signus

Blind PSK/QAM/FSK/Chirp Demodulator — 필사용 코드북

실행에 필요한 코드 전량 · 21개 파일 · 3,520줄 · 총 71쪽 | 2026-08-21 · fe6c42d+수정중

한 줄이 100칸을 넘으면 왼쪽 줄번호 자리에 ↳ 표시가 붙고 아랫줄로 이어집니다 — 원래는 한 줄이니 붙여서 입력하세요. 세로 점선은 들여쓰기 4칸 눈금입니다.

```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "signus"
version = "2.0.0"
description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
requires-python = ">=3.11"
dependencies = ["numpy>=1.26", "scipy>=1.11"]

[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.4"]
# 필사 인쇄물 발급기(tools/codebook.py) 전용. 분석기 본체는 여전히 numpy+scipy 둘뿐이다.
tools = ["pygments>=2.17"]
# Optional JIT for the sequential equalizer/carrier loops (16–34x on those loops).
# Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
fast = ["numba>=0.60"]

[project.scripts]
signus = "signus.cli:main"

[tool.setuptools.packages.find]
include = ["signus*"]

[tool.setuptools.package-data]
signus = ["web/*"]   # the served frontend: without this a pip install 404s the whole UI

[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
addopts = "-q"
markers = ["stretch: slow/marginal cases, opt-in"]

[tool.ruff]
target-version = "py311"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]
```

```python
"""signus — blind signal demodulator with automatic parameter detection."""

__version__ = "2.0.0"
```

```python
"""Sample file I/O. Filename carries read-necessities ONLY (fs, cplx|real, sample
type, endian, bitrev); ground truth lives in a `<filename>.json` sidecar the
analyzer never reads. SigMF sidecars (`<stem>.sigmf-meta`) are also understood.
Endianness falls back to SIGNUS_ENDIAN when the filename carries no .be/.le token."""

import json
import os
import re
from dataclasses import dataclass

import numpy as np

# fs/rf 토막은 옛 밑줄 형식 전용이다. 점으로도 끊길 수 있어 경계에 둘 다 넣는다 -- 새 형식은
# 샘플레이트를 접두사 없이 cplx|real 바로 뒤에 놓으므로 이 정규식에 걸리지 않는다.
_FS_TOK = re.compile(r"(?:^|[._])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)", re.I)
_RF_TOK = re.compile(r"(?:^|[._])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)", re.I)

# token -> (numpy code, full-scale divisor, offset). u-types are offset binary
# (e.g. 8o / RTL-SDR u8: 0..255 with 128 = zero).
_DTYPES = {
    "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
    "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
    "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
}
# baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
_ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
_TOK = {v: k for k, v in _ALIAS.items()}     # 저장할 땐 baudline 표기로 되돌린다
_FMT = {"cplx": "iq", "real": "real", "iq": "iq"}   # iq = 옛 밑줄 형식의 이름
_BITREV = np.array([int(f"{i:08b}"[::-1], 2) for i in range(256)], dtype=np.uint8)

# SigMF core:datatype -> (dtype token, fmt, endian)
_SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
          "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}


@dataclass
class Meta:
    fs: float | None = None
    fmt: str | None = None      # 'iq' | 'real'
    dtype: str = "i16"          # key of _DTYPES
    endian: str = "le"          # 'le' | 'be' (floats included)
    bitrev: bool = False        # bit order reversed within each byte
    rf_center: float | None = None  # RF centre freq (Hz); None = unknown (report baseband)

    def ok(self) -> bool:
        return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES


def parse_name(name: str) -> Meta:
    """파일명이 읽기 필수 정보를 실어 나른다 (정답은 사이드카에만 있고 절대 읽지 않는다).

        새 형식  <이름>.cplx|real.<샘플레이트 정수>.<16t>[.be][.bitrev].pcm
        옛 형식  <이름>_fs<샘플레이트>_iq|real_<i16>[_be][_bitrev].iq

    샘플레이트는 새 형식에서 접두사 없이 온다 -- 그래서 '숫자처럼 생긴 토막'이 아니라
    cplx|real **바로 다음 자리**로 찾는다. 이름 자체가 숫자여도(20260801.cplx.…) 안 헷갈린다.
    확장자를 떼지 않고 통째로 쪼갠다: 새 형식은 마지막 토막(.pcm)이 포맷을 뜻하지 않고,
    옛 형식은 확장자(.iq)가 이미 다른 토막과 같은 말을 하므로 어느 쪽도 손해가 없다."""
    base = os.path.basename(name).lower()
    toks = re.split(r"[._]", base)
```

```python
    # 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 **마지막** 후보. 세 오독을 막는다:
    #  · 라벨의 real/cplx/iq(rec_iq_20260801.cplx.…)가 fs/fmt 를 강탈 -- 마지막 후보 규칙이 막고,
    #  · 점 낀 샘플레이트(cap.cplx.2.4e6.… -> fs=2 Hz)와 라벨 숫자를 fs 로 발명(…_iq_20260728.raw)
    #    -- 숫자 뒤가 제원 토막(16t/be/…/pcm)이어야 한다는 관문이 막는다. 관문에 걸리면 fs 없음으로
    #    크게 죽는다: 조용한 오답 대신 시끄러운 실패가 이 저장소의 원칙이다.
    cands = [k for k, t in enumerate(toks) if t in _FMT]
    hits = [k for k in cands if k + 2 < len(toks) and toks[k + 1].isdigit()
            and (toks[k + 2] in _DTYPES or toks[k + 2] in _ALIAS
                 or toks[k + 2] in ("be", "bitrev", "pcm") or toks[k + 2].startswith("rf"))]
    i = hits[-1] if hits else (cands[0] if cands else None)
    m = Meta()
    if i is not None:                    # 'is not None' 이어야 한다 -- 0 번 토막(cplx.…)도 유효하다
        m.fmt = _FMT[toks[i]]
        if i in hits:
            m.fs = float(toks[i + 1])
    if m.fs is None and (mt := _FS_TOK.search(base)):
        m.fs = float(mt.group(1))
    if rt := _RF_TOK.search(base):
        m.rf_center = float(rt.group(1))
    tail = toks[i + 1:] if i is not None else toks   # 제원은 포맷 토막 **뒤**에만 산다 --
    m.dtype = next((_ALIAS.get(t, t) for t in tail   # 라벨의 u8/f32/be 가 제원을 오염 못 하게
                    if t in _DTYPES or t in _ALIAS), "i16")
    # 엔디안: 파일명 토큰(.be/.le)이 최우선, 없으면 SIGNUS_ENDIAN, 그것도 없으면 le.
    # 실장비 녹음기는 BE 라 장비에서는 export SIGNUS_ENDIAN=be 한 줄로 전 파일이 읽힌다
    # (2026-08-21: BE 캡처를 LE 로 읽어 균등분포 잡음으로 보이던 사고 뒤에 넣었다).
    env = os.environ.get("SIGNUS_ENDIAN", "le").lower()
    if env not in ("le", "be"):
        raise ValueError(f"SIGNUS_ENDIAN 은 le 또는 be 여야 합니다: {env!r}")
    m.endian = "be" if "be" in tail else ("le" if "le" in tail else env)
    m.bitrev = "bitrev" in tail
    return m


def parse_sigmf(path: str) -> Meta | None:
    """Read `<stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
    try:
        with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
            doc = json.load(fh)
        g = doc["global"]
        code = g["core:datatype"].replace("_le", "").replace("_be", "")
        dtype, fmt = _SIGMF[code]
        endian = "be" if g["core:datatype"].endswith("_be") else "le"
        fs = float(g["core:sample_rate"])
    except (OSError, KeyError, ValueError, TypeError, AttributeError):
        return None  # MANDATORY fields malformed: fall back to filename tokens
    try:
        # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
        # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
        # tokens, which can lie about fs (a quiet garbage decode).
        caps = doc.get("captures") or []
        rf = caps[0].get("core:frequency") if caps else None
        rf = None if rf is None else float(rf)
    except (KeyError, ValueError, TypeError, AttributeError, IndexError):
        rf = None
    return Meta(fs, fmt, dtype, endian, rf_center=rf)


def make_name(label: str, m: Meta) -> str:
    """저장은 새 형식만: <이름>.cplx|real.<샘플레이트>.<16t>[.be][.bitrev].pcm"""
    extra = (".be" if m.endian == "be" else "") + (".bitrev" if m.bitrev else "")
```

```python
    kind = "real" if m.fmt == "real" else "cplx"
    return f"{label}.{kind}.{m.fs:.0f}.{_TOK.get(m.dtype, m.dtype)}{extra}.pcm"


def decode(raw: bytes, meta: Meta) -> np.ndarray:
    """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
    Tolerates truncated files (drops trailing partial samples)."""
    code, scale, off = _DTYPES[meta.dtype]
    if meta.bitrev:
        raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
    step = np.dtype(code).itemsize
    dt = ("<" if meta.endian == "le" else ">") + code
    x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
    x = (x - off) / scale
    if meta.fmt == "iq":
        x = x[: x.size & ~1]  # drop dangling half-sample
        return x[0::2] + 1j * x[1::2]
    return x


def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
    meta = meta or parse_sigmf(path) or parse_name(path)
    if not meta.ok():
        raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
    with open(path, "rb") as fh:
        x = decode(fh.read(), meta)
    if x.size < 256:
        raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
    return x, meta


def write(path: str, x: np.ndarray, meta: Meta) -> None:
    """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
    code, scale, off = _DTYPES[meta.dtype]
    flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real
    if scale != 1.0:  # integer types
        peak = np.percentile(np.abs(flat), 99.9) or 1.0
        flat = np.round(flat * (scale - 1) / peak) + off
        info = np.iinfo(code)
        flat = np.clip(flat, info.min, info.max)
    out = flat.astype(("<" if meta.endian == "le" else ">") + code)
    if meta.bitrev:
        out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
                            dtype=out.dtype)
    out.tofile(path)


def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
    doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
    with open(path + ".json", "w") as fh:
        json.dump(doc, fh, indent=1)


def sidecar_read(path: str) -> dict | None:
    try:
        with open(path + ".json") as fh:
            return json.load(fh)
    except (OSError, ValueError):
        return None
```

```python
"""Constellations, Gray labels, differential tables, FSK levels (shared reference)."""

import numpy as np

MODS = ("bpsk", "qpsk", "8psk", "16qam", "64qam")          # blind-classify targets
FSK_MODS = ("fsk2", "fsk4", "msk")
DIFF_MODS = ("dbpsk", "dqpsk", "pi4dqpsk")
GEN_MODS = MODS + ("32qam", "oqpsk") + DIFF_MODS + FSK_MODS  # generator vocabulary

# (data-alphabet order, constellation rotational symmetry). pi4dqpsk: 4 transition
# values per symbol but an 8-point composite constellation.
_INFO = {"bpsk": (2, 2), "qpsk": (4, 4), "8psk": (8, 8), "16qam": (16, 4),
         "64qam": (64, 4), "32qam": (32, 4), "oqpsk": (4, 4), "pi4dqpsk": (4, 8),
         "dbpsk": (2, 2), "dqpsk": (4, 4), "fsk2": (2, 0), "fsk4": (4, 0), "msk": (2, 0)}

# differential phase increment per Gray symbol value
DIFF_PHASES = {
    "dbpsk": np.array([0.0, np.pi]),
    "dqpsk": np.array([0.0, np.pi / 2, np.pi, -np.pi / 2]),
    "pi4dqpsk": np.array([np.pi / 4, 3 * np.pi / 4, -3 * np.pi / 4, -np.pi / 4]),
}


def family(mod: str) -> str:
    return "fsk" if mod in FSK_MODS else "linear"


def mod_order(mod: str) -> int:
    return _INFO[mod][0]


def mod_symmetry(mod: str) -> int:
    """Lowest power p for which x**p strips the modulation (square QAM: 4)."""
    return _INFO[mod][1]


def ideal_points(mod: str) -> np.ndarray:
    if mod in ("bpsk", "dbpsk"):
        return np.array([1.0 + 0j, -1.0 + 0j])
    if mod in ("qpsk", "oqpsk", "dqpsk"):
        return np.exp(1j * (np.pi / 4 + np.arange(4) * np.pi / 2))
    if mod in ("8psk", "pi4dqpsk"):
        return np.exp(2j * np.pi * np.arange(8) / 8)
    if mod == "32qam":  # 6x6 cross: grid minus the four corners
        lv = np.arange(6) * 2.0 - 5
        i, q = np.meshgrid(lv, lv)
        pts = (i + 1j * q).ravel()
        pts = pts[np.abs(pts.real * pts.imag) != 25]
        return pts / np.sqrt(np.mean(np.abs(pts) ** 2))
    n = 4 if mod == "16qam" else 8
    lv = np.arange(n) * 2.0 - (n - 1)
    i, q = np.meshgrid(lv, lv)
    pts = (i + 1j * q).ravel()
    return pts / np.sqrt(np.mean(np.abs(pts) ** 2))


def _gray(k: np.ndarray) -> np.ndarray:
    return k ^ (k >> 1)
```

```python
def bit_labels(mod: str) -> np.ndarray:
    """Gray label per ideal_points index (PSK: phase Gray; square QAM: per-axis).
    32qam uses plain index labels (cross grid has no clean per-axis Gray)."""
    m = mod_order(mod)
    k = np.arange(m)
    if mod in ("16qam", "64qam"):
        n = 4 if mod == "16qam" else 8
        return _gray(k // n) * n + _gray(k % n)
    if mod == "32qam":
        return k
    return _gray(k)


def fsk_levels(mod: str) -> tuple[np.ndarray, np.ndarray]:
    """(frequency levels, Gray label per level) — e.g. fsk4: [-3,-1,1,3], [0,1,3,2]."""
    m = mod_order(mod)
    return np.arange(m) * 2.0 - (m - 1), _gray(np.arange(m))


def to_bits(labels: np.ndarray, mod: str) -> np.ndarray:
    """Integer labels -> flat 0/1 array, MSB first, log2(M) bits per label."""
    k = mod_order(mod).bit_length() - 1
    return (labels[:, None] >> np.arange(k - 1, -1, -1) & 1).ravel().astype(np.uint8)


def demap_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
    """Hard-decide to nearest ideal point, return Gray bits."""
    pts = ideal_points(mod)
    idx = np.argmin(np.abs(symbols[:, None] - pts[None, :]), axis=1)
    return to_bits(bit_labels(mod)[idx], mod)


def demap_diff_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
    """Differential demap: quantize successive phase differences to the mod's
    transition set. Rotation-ambiguity-free (no absolute phase reference)."""
    ph = DIFF_PHASES[mod]
    d = np.angle(symbols[1:] * np.conj(symbols[:-1]))
    err = np.angle(np.exp(1j * (d[:, None] - ph[None, :])))  # wrapped distance
    return to_bits(np.argmin(np.abs(err), axis=1), mod)
```

```python
"""Spectrum + waterfall views of the burst (display only, never used for detection)."""

import numpy as np
from scipy.signal import welch


def _db(p: np.ndarray) -> np.ndarray:
    return 10 * np.log10(p + 1e-20)


def spectrum(x: np.ndarray, fs: float, bins: int = 256) -> dict:
    """Averaged PSD of the burst, decimated to `bins` points. Frequencies in kHz."""
    nper = int(min(4096, max(64, x.size // 8)))
    f, p = welch(x, fs=fs, nperseg=nper, return_onesided=False, detrend=False)
    f, p = np.fft.fftshift(f), _db(np.fft.fftshift(p))
    if f.size > bins:  # max-pool so narrow tones survive decimation
        k = f.size // bins
        f, p = f[: k * bins].reshape(bins, k).mean(1), p[: k * bins].reshape(bins, k).max(1)
    return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}


def waterfall(x: np.ndarray, fs: float, rows: int = 72, bins: int = 144) -> dict:
    """Time-frequency magnitude (dB), row-major, robustly scaled by the caller."""
    nfft = 1 << int(np.ceil(np.log2(max(bins * 2, 64))))
    step = max(1, (x.size - nfft) // max(rows - 1, 1))
    idx = np.arange(rows) * step
    idx = idx[idx + nfft <= x.size]
    if idx.size == 0:
        return {"rows": 0, "bins": bins, "db": [], "dt": 0.0}
    win = np.hanning(nfft)
    seg = np.stack([x[i:i + nfft] * win for i in idx])
    s = np.abs(np.fft.fftshift(np.fft.fft(seg, axis=1), axes=1)) ** 2
    k = nfft // bins
    s = s[:, : k * bins].reshape(idx.size, bins, k).mean(2)
    return {"rows": int(idx.size), "bins": bins, "dt": float(step / fs),
            "db": np.round(_db(s).ravel(), 2).tolist()}
```

```python
"""Wideband signal detection: find every emitter in a capture as a frequency/time
box. Turns signus from a single-signal demodulator into a survey tool. Frequency
detection runs on the time-averaged spectrogram (robust to signal width); the time
extent per band comes from the spectrogram. numpy + scipy only."""

from dataclasses import dataclass

import numpy as np
from scipy import ndimage
from scipy.signal import get_window, stft


@dataclass
class Detection:
    fc: float          # centre frequency (Hz, baseband offset)
    bw: float          # 99%-power bandwidth (Hz)
    t0: int            # burst start / end (sample indices into the capture)
    t1: int
    snr_db: float      # peak-to-floor of the band (dB)

    @property
    def baud_hint(self) -> float:
        """Rough symbol-rate prior: 99%-power BW is ~0.86 of occupied BW Rs(1+a)."""
        return self.bw / (0.86 * 1.25)


def _floor(s: np.ndarray, frac: float = 0.5, pct: int = 20) -> np.ndarray:
    """Per-bin noise floor via a wide low-percentile filter. A 20th percentile over
    a half-band window tracks the front-end tilt yet stays on the noise even when a
    signal fills up to ~half the window -- so it survives both a narrowband emitter
    and a single signal occupying 10-50% of the band."""
    w = max(11, int(frac * s.size) | 1)
    return ndimage.percentile_filter(s, pct, size=w, mode="nearest")


def detect(x: np.ndarray, fs: float, *, nfft: int = 4096, overlap: float = 0.5,
           thr_db: float = 6.0, min_bw_bins: int = 3) -> list[Detection]:
    """Detect emitters as (fc, bw, t0, t1) boxes, time/frequency ordered by fc.
    Empty capture -> []; a single signal filling the band -> one whole-band box."""
    if x.size == 0:
        return []
    # nfft never exceeds the record (else stft rejects an over-long window)
    nfft = int(min(nfft, x.size, 1 << max(8, int(np.log2(max(x.size // 16, 256))))))
    win = get_window("blackmanharris", nfft)
    f, t, z = stft(x, fs=fs, window=win, nperseg=nfft, noverlap=int(nfft * overlap),
                   return_onesided=False, boundary=None, padded=False)
    f = np.fft.fftshift(f)
    p = np.fft.fftshift(np.abs(z) ** 2, axes=0)          # (freq, time) linear power
    s = p.mean(axis=1)                                     # time-averaged PSD
    floor = _floor(s)
    mask = s > floor * 10 ** (thr_db / 10)
    close = max(3, int(0.012 * s.size) | 1)              # bridge intra-signal nulls
    mask = ndimage.binary_opening(mask, structure=np.ones(2))
    mask = ndimage.binary_closing(mask, structure=np.ones(close))

    lab, n = ndimage.label(mask)
    regions = [np.where(lab == i)[0] for i in range(1, n + 1)]
    regions = [r for r in regions if r.size >= min_bw_bins]
    merged: list[np.ndarray] = []
    for r in sorted(regions, key=lambda r: r[0]):
```

```
        # heal an intra-signal split (gap small vs the signal's OWN width) without
        # swallowing a genuinely separate neighbour -- cap the merge gap at half the
        # narrower blob, so adjacent channels (e.g. the 25 kHz marine raster) survive
        if merged and r[0] - merged[-1][-1] <= max(close, min(merged[-1].size, r.size) // 2):
            merged[-1] = np.arange(merged[-1][0], r[-1] + 1)
        else:
            merged.append(r)
    if len(merged) >= 2:  # frequency is circular: heal a signal straddling +-fs/2
        wrap_gap = (s.size - 1 - merged[-1][-1]) + merged[0][0]
        if wrap_gap <= max(close, min(merged[0].size, merged[-1].size) // 2):
            merged[0] = np.concatenate([merged.pop(), merged[0]])  # wraps the edge

    dets = [_measure(r, s, floor, p, f, t, fs, x.size) for r in merged]
    if not dets and s.max() > 10 ** (thr_db / 10) * np.percentile(s, 20):
        # floor defeated by a signal filling most of the band: one whole-band box,
        # so survey() falls back to analysing the entire capture (legacy behaviour)
        dets = [Detection(0.0, fs * 0.5, 0, x.size, 0.0)]
    return dets


def _measure(r: np.ndarray, s: np.ndarray, floor: np.ndarray, p: np.ndarray,
             f: np.ndarray, t: np.ndarray, fs: float, n: int) -> Detection:
    """Estimate fc (floor-subtracted centroid), 99%-power bw, and time extent.
    Handles a wrapped index set (a signal straddling +-fs/2, non-contiguous)."""
    wrapped = r.size > 1 and not np.all(np.diff(r) == 1)
    if wrapped:
        idx = r                                         # already spans the edge
        fr = f[idx].astype(float).copy()
        fr[idx < s.size // 2] += fs                     # lift the -fs/2 side onto a line
    else:
        idx = np.arange(max(0, r[0] - 3), min(s.size, r[-1] + 4))  # widen for skirts
        fr = f[idx]
    wgt = np.maximum(s[idx] - floor[idx], 0)
    fc = float(np.sum(fr * wgt) / (np.sum(wgt) + 1e-30))
    o = np.argsort(fr)
    cs = np.cumsum(wgt[o]) / (np.sum(wgt) + 1e-30)
    f_lo = fr[o][np.searchsorted(cs, 0.005)]
    f_hi = fr[o][min(np.searchsorted(cs, 0.995), idx.size - 1)]
    bw = float(abs(f_hi - f_lo))
    if fc >= fs / 2:                                    # fold a wrapped centroid back
        fc -= fs
    band = p[idx, :].sum(axis=0)                        # time profile of the band
    on = np.where(band > max(np.median(band) * 2.0, band.max() * 0.1))[0]
    t0 = int(t[on[0]] * fs) if on.size else 0
    t1 = int(t[min(on[-1], t.size - 1)] * fs) if on.size else n
    snr = float(10 * np.log10(s[idx].max() / (np.median(floor) + 1e-30)))
    return Detection(fc, bw, t0, t1, snr)
```

```python
"""Channel extraction: pull one detected emitter out of a wideband capture as a
baseband stream, sized so the signal occupies ~1/4-1/3 of the reduced rate -- the
empirically-validated sweet spot for the downstream blind estimators (fill the band
and the matched filter starves; leave it too wide and noise dominates)."""

import numpy as np
from scipy.signal import firwin, resample_poly

from . import dsp
from .detect import Detection


def extract(x: np.ndarray, fs: float, det: Detection, *, target_frac: float = 0.28,
            pad: float = 1.5, ntaps: int = 129) -> tuple[np.ndarray, float]:
    """Mix det.fc to DC, low-pass to the channel, decimate; trim to the burst.
    Returns (baseband IQ, channel sample rate)."""
    bb = dsp.mix(x, fs, det.fc)
    want = max(det.bw * pad, fs * 1e-4)                  # passband to keep
    d = max(1, int(fs / (want / target_frac)))          # want ~= target_frac * fs/d
    fs_ch = fs / d
    # Low-pass to the box ALWAYS, even when d==1 (a wide box, bw > ~0.28*fs): otherwise the
    # "channel" is the whole capture merely mixed, and analyze locks onto a DIFFERENT, stronger
    # emitter (reported as a confident duplicate). Decimate only when there is room to.
    cutoff = min(0.9 * fs_ch / 2, want / 2) / (fs / 2)
    bb = np.convolve(bb, firwin(ntaps, cutoff), "same")
    if d > 1:
        bb = resample_poly(bb, 1, d)
    a, b = int(det.t0 // d), int(det.t1 // d)
    if b - a >= 256:                                     # trim empty time, keep guard
        g = (b - a) // 20
        bb = bb[max(0, a - g):b + g]
    return bb.astype(np.complex128), fs_ch
```

```python
"""Per-channel triage: decide whether an extracted channel is a demodulable digital
signal or something the demodulator must NOT force onto a constellation (analog FM
voice, a CW tone/spur). Envelope CV is the SNR-robust separator: RRC-shaped PSK/QAM
always sit at CV>=0.25, while FSK/FM/CW are constant-envelope (CV<=0.05)."""

import numpy as np

from .chirp import is_chirp, sweeps_band
from .fsk import fsk_gate

_CV_CE = 0.15      # constant-envelope ceiling: below it a non-FSK signal is analog/tone
_TONE_PAR = 500.0  # M-th-power peak-to-mean above which a constant-envelope signal is CW


def _carrier_par(x: np.ndarray, powers: tuple[int, ...] = (1, 2, 4, 8)) -> float:
    """Best peak-to-mean of the M-th-power magnitude spectrum. A CW tone spikes at
    p=1; analog FM and noise stay flat at every power."""
    x = x - x.mean()
    nfft = 1 << int(np.ceil(np.log2(x.size)))
    win = np.hanning(x.size)
    best = 0.0
    for p in powers:
        spec = np.abs(np.fft.fft(x ** p * win, nfft))
        best = max(best, float(spec.max() / (spec.mean() + 1e-30)))
    return best


def _active(x: np.ndarray) -> np.ndarray:
    """Trim leading/trailing dead air (envelope below half the top-quartile median). A bursty
    channel's dead air corrupts BOTH triage discriminators -- it inflates the envelope CV past
    fsk_gate's ceiling AND pollutes sweeps_band's IF histogram -- so every gate below must see
    the emitter, not the silence around it. A continuous channel is returned unchanged."""
    env = np.abs(x)
    top = env[env > np.percentile(env, 75)]
    if top.size == 0:
        return x
    on = np.flatnonzero(env > 0.5 * np.median(top))
    if on.size < 256:
        return x
    return x[on[0]:on[-1] + 1]


def family(x: np.ndarray, fs: float) -> str:
    """One of 'fsk' | 'chirp' | 'linear' | 'analog' | 'tone'. 'linear'/'fsk' go to the
    demod; 'chirp'/'analog'/'tone' are reported as-is (never force-fit to a constellation)."""
    x = _active(x)
    if fsk_gate(x, fs):
        # a linear FMCW chirp / CSS (LoRa) trips fsk_gate too -- its swept IF reads bimodal to
        # the gate -- and would then be force-demodulated into confident garbage FSK symbols.
        # is_chirp fires on both a chirp and (at some h/baud) real FSK, so the IF-SWEEP test
        # breaks the tie: a genuine band-sweep is characterized as chirp, real FSK stays FSK.
        if is_chirp(x, fs) and sweeps_band(x, fs):
            return "chirp"
        return "fsk"
    if is_chirp(x, fs) and sweeps_band(x, fs):
        # linear chirp / CSS (LoRa) -- constant-envelope, would otherwise fall through to
        # 'analog' (no M-power tone). The tie-break is REQUIRED here too: a bursty integer-h
        # FSK channel (dead air inflates envelope CV past fsk_gate) also trips is_chirp via
        # its CPFSK beat tones -- without sweeps_band it was labeled 'chirp' and never
        # demodulated. A non-sweeping channel falls through, and the demod's burst isolation
```

```python
        # recovers the FSK.
        return "chirp"
    a = np.abs(x)
    if a.std() / (a.mean() + 1e-12) < _CV_CE:            # constant envelope, not FSK
        return "tone" if _carrier_par(x) >= _TONE_PAR else "analog"
    return "linear"
```

```python
"""Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
anti-shared-bug rule this imports only constellation reference data, never gen.py."""

from fractions import Fraction

import numpy as np
from scipy.ndimage import uniform_filter1d
from scipy.signal import get_window, hilbert, resample_poly, stft, welch

from ._accel import dd_carrier
from .constellations import ideal_points


def analytic(x: np.ndarray) -> np.ndarray:
    """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
    samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
    overflows on the |x|^2 the estimators take)."""
    x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
    x = np.where(np.abs(x) > 1e150, 0.0, x)   # a finite spike that overflows |x|^2 is corrupt too
    if np.iscomplexobj(x):
        return x.astype(np.complex128)
    return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)


def _parab(ydb: np.ndarray, k: int) -> float:
    """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
    if k <= 0 or k >= ydb.size - 1:
        return 0.0
    a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
    d = a - 2 * b + c
    # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
    # (d~=0) or non-max triple can explode the ratio and drive baud negative -> a resample crash.
    return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0


def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
    """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
    Assignment is a running min over the k centers (no (N, k) broadcast temporary);
    strict `<` keeps the lowest-index center on ties, so labels are identical to a
    broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
    c = np.quantile(a, (np.arange(k) + 0.5) / k)
    lab = np.zeros(a.size, dtype=int)
    for _ in range(iters):
        d = np.abs(a - c[0])
        lab = np.zeros(a.size, dtype=int)
        for j in range(1, k):
            dj = np.abs(a - c[j])
            closer = dj < d
            lab[closer] = j
            d[closer] = dj[closer]
        nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
        if np.allclose(nc, c):
            break
        c = nc
    return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))


# --- burst detection -------------------------------------------------------

_SPIKE_PAR = 60.0  # raw peak-to-mean power above which a candidate run is an impulse smear, not a
```

```
#                       burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/QAM
#                       peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared over
#                       a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with margin.


def _otsu(v: np.ndarray, bins: int = 128) -> float:
    """Otsu split threshold of a 1-D value distribution (log powers, column scores)."""
    hist, edges = np.histogram(v, bins=bins)
    centers = (edges[:-1] + edges[1:]) / 2
    w = np.cumsum(hist).astype(float)
    m = np.cumsum(hist * centers)
    mb = m / np.where(w > 0, w, 1)
    mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
    return float(centers[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])


def _cell_power(x: np.ndarray, fs: float, nperseg: int = 256) -> tuple[np.ndarray, int, int]:
    """Waterfall cell power for detection: (P[fbin, col], hop, nperseg). nperseg
    auto-shrinks on short records; shared core for find_bursts (and, later, survey)."""
    nperseg = int(min(nperseg, max(64, 1 << int(np.log2(max(x.size // 2, 64))))))
    hop = nperseg // 2
    win = get_window("blackmanharris", nperseg)
    _, _, z = stft(x, fs=fs, window=win, nperseg=nperseg, noverlap=nperseg - hop,
                   return_onesided=False, boundary=None, padded=False)
    return np.abs(z) ** 2, hop, nperseg


def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
    """Waterfall (cell-level) burst detector. Per-bin noise floors give the same
    narrowband processing gain the operator's eye gets on a spectrogram (a burst at
    wideband 0 dB is +12 dB per cell when it occupies 1/16 of the band); column
    scores then drive the run/merge/guard machinery on the time axis.
    Returns bursts in time order, or [(0, size)] when nothing stands out."""
    n = x.size
    pw = np.abs(x) ** 2
    # A burst is a TIME-ENERGY event. A continuous signal that merely shuffles its spectrum
    # (multi-tone FSK) lights different cells per column and once shattered into 78 phantom
    # bursts -- but its wideband envelope is FLAT. Envelope Otsu separation measured 0.014-
    # 0.052 decades on continuous FSK vs >=0.30 on every real burst regime: nothing turns
    # on or off -> the whole record is the burst.
    lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
    e_thr = _otsu(lp)
    e_lo, e_hi = lp[lp < e_thr], lp[lp >= e_thr]
    if not e_lo.size or not e_hi.size or float(e_hi.mean() - e_lo.mean()) < 0.12:
        return [(0, n)]
    # nperseg 128: bin 7.8 kHz keeps the narrowband gain for real signal widths while the
    # 64-sample hop resolves inter-burst gaps down to ~200 samples (256 could not split a
    # 300-sample gap -- no column fits inside it).
    P, hop, nperseg = _cell_power(x, fs, nperseg=128)
    if P.shape[1] < 4:                  # too few columns to tell bursts from record
        return [(0, n)]
    # Per-bin floor at a LOW percentile: tracks coloured noise bin by bin and stays on the
    # noise even when a bin is signal-occupied up to ~90% of the time (high-duty trains).
    # A record-filling signal owns its bins' floors entirely -> flat score -> [(0, n)].
    floor_f = np.percentile(P, 10, axis=1)[:, None]
    r = P / (floor_f + 1e-30)
    k = max(3, P.shape[0] // 16)
    sc = np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30)  # column score (decades)
    # A single noise column can spike +0.53 decades over base -- OVERLAPPING the weakest
    # real bursts (+0.57). Like the eye, the discriminator is PERSISTENCE, not height: a
```

```python
    # 3-column smooth drops the noise worst-case to 0.31-0.33 while a real burst (>=3
    # columns) keeps its level (0.56+) -- the gap the fixed bars below live in.
    sc = uniform_filter1d(sc, 3)
    # The noise base is the LOWEST MODE of the score histogram (Otsu, as the wideband
    # version proved out): fixed quantile anchors cannot serve every duty regime, and
    # spread ESTIMATES broke three ways (median blind >50% duty, two-sided MAD inflated by
    # skirts, one-sided by the score's left tail). The margins are FIXED instead: a noise
    # column's score is distribution-pinned at C = log10((ln(nbins/k)+1)/0.105) whatever
    # the noise level, so its fluctuation is a constant of (nbins, k) -- measured worst
    # 0.33 over 24 noise records vs 0.56 for the weakest keeper burst.
    thr = _otsu(sc, bins=64)
    quiet = sc[sc < thr]
    if not quiet.size:
        return [(0, n)]
    base = float(np.median(quiet))
    # No noise-quiet column at all (base far above the pinned C): a continuous signal
    # shuffling its spectrum (a 4-tone FSK read as 78 phantom bursts before this guard).
    c_noise = float(np.log10((np.log(P.shape[0] / k) + 1) / 0.105))
    if base > c_noise + 0.25:
        return [(0, n)]
    hi = base + 0.45
    lo = base + 0.25
    above_lo = sc >= lo
    dif = np.diff(above_lo.astype(np.int8))
    starts = list(np.where(dif == 1)[0] + 1)
    ends = list(np.where(dif == -1)[0] + 1)
    if above_lo[0]:
        starts.insert(0, 0)
    if above_lo[-1]:
        ends.append(sc.size)
    runs = [(s, e) for s, e in zip(starts, ends, strict=True) if (sc[s:e] >= hi).any()]
    if not runs:
        return [(0, n)]
    merged = [list(runs[0])]
    for s, e in runs[1:]:
        # A real inter-burst gap returns to the score base; a marginal-level burst dips just
        # below lo without reaching it and would otherwise shatter into fragments. Merge
        # across any valley whose median stays off the base.
        valley = sc[merged[-1][1]:s]
        if s - merged[-1][1] < 2 or float(np.median(valley)) > base + 0.12:
            merged[-1][1] = e
        else:
            merged.append([s, e])
    # Shape gate: a modulated burst has a near-flat raw envelope (peak/mean ~ a few); an
    # impulse smear is one spike over noise (ratio ~span). Reject spike-dominated candidates.
    out, spiked = [], 0
    for c0, c1 in merged:
        if c1 - c0 < 2:                  # single-column flicker (noise / impulse smear)
            continue
        s, e = max(0, c0 * hop), min(n, (c1 - 1) * hop + nperseg)
        if c1 - c0 >= 6:
            # long bursts: trim one hop of smear/quantization slack per edge. The noise
            # tail otherwise fed to the demod sits on a knife edge for dense QAM (58 extra
            # leading samples flipped a 16qam@24dB slice to 64qam lock 18). Short runs keep
            # their full span -- trimming there would eat into the burst itself.
            s, e = s + hop, e - hop
        seg = pw[s:e]
        if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
            spiked += e - s
            continue
```

```python
        out.append((int(s), int(e)))
    # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
    # inside the strongest burst killed its candidate, returning the survivors would hand
    # analyze() the wrong emitter -- fall back instead. Only spike kills count: the
    # single-column drops above are routine flicker filtering.
    covered = sum(e - s for s, e in out)
    if out and covered < 0.6 * (covered + spiked):
        return [(0, n)]
    # A continuous signal OWNS its bins' floors and is invisible to the cell path -- but its
    # edge transients are not, and once returned [(0, 192)] for a 64k channel whose 16qam
    # ran the whole record. If the detections cover almost none of the envelope's high-power
    # mass, the cell path missed the main event: return the ENVELOPE's high span instead
    # (not the raw record -- a dead tail fed to the demod flipped that 16qam to 64qam).
    # 0.1 keeps partial detections (a strong burst next to a lo-only sibling covers ~30%).
    if out and covered < 0.1 * int((lp >= e_thr).sum()):
        idx = np.where(lp >= e_thr)[0]
        return [(int(idx[0]), int(idx[-1]) + 1)]
    return out or [(0, n)]


def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
    """The most energetic burst of find_bursts."""
    return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))


# --- carrier estimation ---------------------------------------------------

def est_carrier(
    x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
    nfft: int = 65536, blocks: int = 4,
) -> tuple[float, int, bool]:
    """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
    nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
    freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
    best_par, best_s, sym = -1.0, None, powers[0]
    for p in powers:
        xp = x ** p
        blk = xp.size // blocks
        segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
        acc = np.zeros(nfft)
        for s in segs:
            acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
        spec = acc / len(segs)
        par = spec.max() / spec.mean()
        if par > best_par:
            best_par, best_s, sym = par, spec, p

    if best_s is None:  # degenerate input (all-zero / no energy): no M-th-power tone exists
        return 0.0, int(powers[0]), True
    ydb = 20 * np.log10(best_s + 1e-20)
    k = int(np.argmax(best_s))
    f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
    fc = f_peak / sym
    ambiguous = abs(sym * fc) > 0.4 * fs
    return float(fc), int(sym), bool(ambiguous)


def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
    """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
    whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
```

```
    (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
    straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
    f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
    p = np.maximum(pxx - np.median(pxx), 0)
    # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
    # 8psk beyond 0.3125*fs with no candidate).
    cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)
             if abs(fc + k * fs / sym) < 0.5 * fs]
    half = fs / (2 * sym)                        # each candidate owns a fs/sym-wide cell
    return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum()))


def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
    """Downconvert by fc (complex heterodyne)."""
    return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)


# --- symbol rate + rolloff -------------------------------------------------

def est_baud(
    x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
    blocks: int = 4,
) -> tuple[float, float]:
    """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
    Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
    a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
    u = np.abs(x) ** 2
    u = u - u.mean()
    blk = u.size // blocks
    segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
    nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs)))))
    acc = np.zeros(nfft // 2 + 1)
    for s in segs:
        acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
    spec = acc / len(segs)
    freqs = np.fft.rfftfreq(nfft, 1 / fs)

    lo = lo if lo is not None else 0.005 * fs
    hi = hi if hi is not None else 0.45 * fs
    band = (freqs >= lo) & (freqs <= hi)
    if not band.any():  # degenerate window (e.g. noise-derived prior): default band
        band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
    idx = np.where(band)[0]
    k = idx[int(np.argmax(spec[idx]))]
    ydb = 20 * np.log10(spec + 1e-20)
    baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
    conf = spec[k] / (np.median(spec[band]) + 1e-20)
    return float(baud), float(conf)


def occupied_bw(x: np.ndarray, fs: float) -> float:
    """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
    from the 25th-percentile floor to the 90th-percentile in-band level, walk
    outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
    Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
    f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
    floor = np.percentile(pxx, 25)
    order = np.argsort(np.abs(f))
    idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
    if idx.size == 0:
```

```python
        return 0.0
    last = idx[-1]
    return float(2 * np.abs(f[order][last]))


def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
    """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
    n = x.size
    nper = min(8192, max(256, n // 8))
    f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
    f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
    af = np.abs(f)
    inband = np.median(pxx[af < 0.25 * baud])
    noise = np.median(pxx[af > 0.9 * baud])
    thr = noise + 0.10 * (inband - noise)
    band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
    if not band.any():
        return 0.35
    raw = 2 * af[band].max() / baud - 1
    return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))


# --- resampling + matched filter ------------------------------------------

def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
    """Rational resample to exactly sps samples/symbol."""
    r = Fraction(sps * baud / fs).limit_denominator(2000)
    return resample_poly(x, r.numerator, r.denominator)


def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
    """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
    |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
    m = (span * sps) // 2
    t = np.arange(-m, m + 1) / sps
    a = alpha
    num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
    den = np.pi * t * (1 - (4 * a * t) ** 2)
    h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
    h[t == 0] = 1 - a + 4 * a / np.pi
    if a > 0:
        s = np.isclose(np.abs(t), 1 / (4 * a))
        h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
                                 + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
    return h / np.sqrt(np.sum(h ** 2))


def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
    """Apply the RX matched RRC filter."""
    return np.convolve(x, _rx_rrc(alpha, span, sps), "same")


# --- timing recovery -------------------------------------------------------

def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
    """Block-wise Oerder-Meyr timing recovery; returns `out` samples/symbol
    (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
    Per block the spectral-line phase gives the offset; unwrapped across blocks
    to track clock drift, then interpolated per symbol."""
    n = sps
```

```python
    nsym = x.size // n
    if nsym < 1:
        return x[:0].astype(np.complex128)
    xt = x[:nsym * n]
    p = np.abs(xt) ** 2
    ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
    bs = block * n
    if xt.size <= bs:  # single block: one global fractional offset
        eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
    else:
        nblk = xt.size // bs
        c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
        eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
        eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
    pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
    idx = np.arange(xt.size)
    return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)


# --- decision-directed carrier sync ----------------------------------------

def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarray:
    """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
    Costas loop, which jitters QAM at the correct points). Sequential feedback (each
    phase update depends on the prior decision) -> numba kernel when available."""
    pts = ideal_points(mod)
    z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2))  # AGC to unit power
    return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
                      np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)
```

```python
"""Blind symbol-spaced FIR equalizer: CMA acquisition then decision-directed LMS,
to undo the multipath ISI (the generator's `taps=` channel) that a phase-only
carrier loop cannot remove."""

import numpy as np

from ._accel import cma_dd_equalize, cma_dd_equalize_fse
from .constellations import ideal_points


def equalize(symbols: np.ndarray, mod: str, n_taps: int = 11, mu_cma: float = 5e-3,
             mu_dd: float = 2e-3, warmup: int = 1500) -> np.ndarray:
    """Blind FIR equalizer for symbol-spaced samples; returns the cleaned symbols.

    Phase 1 is constant-modulus (Godard) acquisition, phase 2 refines by decisions,
    then the converged taps refilter the whole record so early symbols are clean too.
    """
    pts = ideal_points(mod)
    r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2)  # Godard dispersion const
    x = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2))        # AGC to unit power
    # The adaptive passes (every tap update depends on the previous output/decision) are
    # the one part that cannot be vectorized; they run as a numba kernel when available.
    return cma_dd_equalize(np.ascontiguousarray(x, dtype=np.complex128),
                           np.ascontiguousarray(pts, dtype=np.complex128),
                           float(r2), n_taps, mu_cma, mu_dd, warmup)


def equalize_fse(x2: np.ndarray, mod: str, n_taps: int = 75, mu_cma: float = 1e-3,
                 mu_dd: float = 8e-4, warmup: int = 4000) -> np.ndarray:
    """T/2 fractionally-spaced CMA->DD equalizer: 2 samples/symbol in, symbols out.

    Sees the unaliased spectrum, so it inverts channels whose symbol-rate folding
    puts zeros near the unit circle (e.g. a strong 1.5-symbol echo) where the
    symbol-spaced `equalize` cannot. A strong echo's inverse is LONG (0.8 gain
    needs ~70 T/2 taps for -20 dB); the clock must already be tracked, so feed it
    `timing(ym, sps, out=2)` -- fixed taps cannot follow a drifting clock.
    Calibrated on 2ray(0.8, 1.5 sym) x seeds 0-2: lock 77-80 (61 taps: 57-66)."""
    pts = ideal_points(mod)
    r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2)  # Godard dispersion const
    x = x2 / np.sqrt(np.mean(np.abs(x2) ** 2))                  # AGC to unit power
    # Seven sequential passes (2 CMA + 4 DD + 1 tracked output); numba kernel when available.
    return cma_dd_equalize_fse(np.ascontiguousarray(x, dtype=np.complex128),
                               np.ascontiguousarray(pts, dtype=np.complex128),
                               float(r2), n_taps, mu_cma, mu_dd, warmup)
```

```python
"""Blind repeated-preamble synchronization (definition-free).

A real packetized emission begins with a preamble: a short block repeated a few times so a
receiver can lock. The repetition is detectable and exploitable WITHOUT knowing its content --
the delay-and-correlate (Schmidl-Cox) metric M(d) = |Sum conj(x[d+k]) x[d+k+P]|^2 / (Sum |x|^2)^2
rises to ~1 over the span where two period-P windows coincide (inside the preamble), and the PHASE
of that correlation is a low-variance carrier-frequency-offset (CFO) estimate that works from only a
few symbols -- where the blind M-th-power carrier estimator, needing many symbols to average, fails.

Used as a gated, keep-best rescue in pipeline.analyze(): a signal with no preamble yields no
plateau -> no rescue -> the existing blind chain is untouched (sweep stays byte-identical)."""

from dataclasses import dataclass

import numpy as np

_CONF_MIN = 0.80    # plateau strength floor (median M over the run); calibrated vs no-preamble
_RUN_MIN = 4.0      # plateau length >= this * period. RRC pulse memory spans a FIXED ~4.5 symbols,
#                     so in periods it exceeds this only at a 1-2 symbol lag and dies by P~=3 syms;
#                     a real preamble's plateau is (R-1) periods long -> this rejects the short
#                     memory (needs R>=~5 repeats; residual random false-alarms are caught by the
#                     pipeline keep-best-lock gate -- a spurious CFO lowers lock and is discarded).
_HARM_MIN = 0.50    # the SAME plateau must also correlate at lag 2P (a real preamble, >=3 repeats)
_VALLEY_MAX = 0.80  # ...AND DROP at lag 1.5P (between harmonics): a real preamble is peaked at
#                     multiples of P (a valley in between), so median(M@1.5P) < 0.85*conf. RRC pulse
#                     memory is a smooth decay (~equal at P and 1.5P) -> rejected. Peak/valley +
#                     run length is what lets the period be scanned directly, WITHOUT a reliable
#                     baud (est_baud is exactly what fails on the short bursts we target).
_PMIN = 12          # smallest period to scan (samples); below this is intra-symbol pulse memory
_PMAX = 512         # largest period to scan; a preamble block is short (L<=~12 syms, sps<=~40)
_WMAX = 8192        # only the leading window is scanned -- a preamble sits at the burst START


@dataclass
class Preamble:
    start: int      # sample index where the repeated region begins
    end: int        # sample index where the preamble ends (data starts ~here)
    period: int     # repetition period in samples (= L * sps)
    cfo_hz: float   # carrier-frequency offset estimated from the preamble phase slope
    conf: float     # 0..1 plateau confidence (median metric over the detected run)


def _metric(x: np.ndarray, P: int) -> tuple[np.ndarray, np.ndarray]:
    """Normalized delay-correlation M(d) in [0,1] and the raw correlation Pmet(d), lag P, window
    W=P. Normalized by BOTH windows' energy (Cauchy-Schwarz) so M<=1 -- a true correlation
    coefficient, ~1 only where the two period-P windows are genuinely identical (a preamble)."""
    prod = np.conj(x[:-P]) * x[P:]          # conj(x[n]) x[n+P]
    ea = np.abs(x[:-P]) ** 2                # first-window energy
    eb = np.abs(x[P:]) ** 2                 # second-window energy
    cp = np.concatenate([[0.0 + 0j], np.cumsum(prod)])
    ca = np.concatenate([[0.0], np.cumsum(ea)])
    cb = np.concatenate([[0.0], np.cumsum(eb)])
    W = P
    pm = cp[W:] - cp[:-W]                    # Sum prod[d:d+W]
    da = ca[W:] - ca[:-W]
    db = cb[W:] - cb[:-W]
    return np.abs(pm) ** 2 / (da * db + 1e-30), pm


def _seg_median(x: np.ndarray, lag: int, s: int, ln: int) -> float:
```

```python
    """Median of the delay-correlation metric at `lag` over the plateau [s, s+ln)."""
    m = _metric(x, lag)[0]
    e = min(s + ln, m.size)
    return float(np.median(m[s:e])) if e > s else 0.0


def _longest_run(mask: np.ndarray) -> tuple[int, int]:
    """(start, length) of the longest True run in a boolean array; (0, 0) if none."""
    if not mask.any():
        return 0, 0
    d = np.diff(np.concatenate([[0], mask.view(np.int8), [0]]))
    starts = np.where(d == 1)[0]
    ends = np.where(d == -1)[0]
    i = int(np.argmax(ends - starts))
    return int(starts[i]), int(ends[i] - starts[i])


def find_preamble(x: np.ndarray, fs: float, baud_hint: float | None = None) -> Preamble | None:
    """Detect a repeated preamble and estimate its (start, end, period, CFO, confidence).
    Returns None when no repeated block stands out. The period is scanned DIRECTLY (not derived
    from baud) so it works even when est_baud fails on the short burst -- exactly the target case.
    baud_hint, if given, only tightens the scan's upper bound for speed."""
    x = np.asarray(x, dtype=np.complex128)
    x = x - x.mean()
    if x.size < 256 or not np.all(np.isfinite(x)):
        return None
    xw = x[:_WMAX]                           # a preamble sits at the burst start
    n = xw.size
    pmax = _PMAX if not (baud_hint and baud_hint > 0) else min(_PMAX, int(12 * fs / baud_hint))
    pmax = min(pmax, (n - 8) // 3)           # need >=3 windows for the 2P harmonic check
    best = None
    for P in range(_PMIN, max(_PMIN + 1, pmax + 1)):
        M, pm = _metric(xw, P)
        run_start, run_len = _longest_run(M >= _CONF_MIN)
        if run_len < _RUN_MIN * P:
            continue
        conf = float(np.median(M[run_start:run_start + run_len]))
        # peak/valley gate: a genuinely periodic preamble correlates at 2P (harmonic) but DROPS at
        # 1.5P (between harmonics); RRC pulse memory (which fakes a plateau at a 1-2 symbol lag) is
        # a smooth decay -- high at both -> rejected, letting the period be scanned directly.
        harm = _seg_median(xw, 2 * P, run_start, run_len)
        valley = _seg_median(xw, int(round(1.5 * P)), run_start, run_len)
        if harm < _HARM_MIN or valley > _VALLEY_MAX * conf:
            continue
        score = run_len * conf               # long AND strong wins
        if best is None or score > best[0]:
            ang = float(np.angle(pm[run_start:run_start + run_len].sum()))  # coherent, wrap-safe
            cfo = ang * fs / (2 * np.pi * P)
            end = run_start + run_len + 2 * P            # past the last correlated repeat -> data
            best = (score, Preamble(run_start, min(end, x.size), P, cfo, conf))
    return best[1] if best else None
```

```python
"""Decision layer: modulation classification, rotation alignment, lock quality, SNR."""

from dataclasses import dataclass

import numpy as np

from .constellations import ideal_points, mod_symmetry
from .dsp import _kmeans1d

# Amplitude-ring separators, calibrated through the real front-end (seeds 0-3, snr 14-25):
# 16qam <=0.80 and 64qam <=0.99 vs 32qam >=1.16 on s3*s9/s5**2.
_D32 = 1.07
_C42_PSK = 0.80   # constant-modulus 4th cumulant: qpsk ~1.0, QAM <0.7
_CM_TOL = 0.2     # 1-ring spread guard for constant-modulus signals
_R39 = 2.7        # s3/s9: 16qam ~2.1, 64qam >=3.0


def _agc(z: np.ndarray) -> np.ndarray:
    """Scale to unit average power."""
    return z / np.sqrt(np.mean(np.abs(z) ** 2))


def _c42(z: np.ndarray) -> float:
    """Magnitude of the normalized 4th-order cumulant C42 (rotation invariant)."""
    z = _agc(z)
    p = np.mean(np.abs(z) ** 2)
    return float(abs(np.mean(np.abs(z) ** 4) - abs(np.mean(z**2)) ** 2 - 2 * p**2))


def _spread(a: np.ndarray, k: int) -> float:
    """RMS within-cluster spread from the shared 1-D k-means (sorted-value ring test)."""
    return _kmeans1d(a, k)[2]


def classify(z: np.ndarray, symmetry: int) -> str:
    """Symmetry 2->bpsk, 8->8psk; 4-> qpsk/16qam/32qam/64qam via C42 then amplitude rings.

    Not blind classes (documented): oqpsk collides with square-QAM on every ring/cumulant
    feature; pi4dqpsk's 4th power carries a pi-per-symbol rotation, so the carrier estimate
    absorbs baud/8 and it arrives here looking exactly like qpsk (its bits still decode
    correctly via differential demapping).
    """
    if symmetry == 2:
        return "bpsk"
    if symmetry == 8:
        return "8psk"
    z = _agc(z)
    if _c42(z) >= _C42_PSK:
        return "qpsk"
    a = np.sort(np.abs(z))
    s1 = _spread(a, 1)
    if s1 <= _CM_TOL:  # constant-modulus guard (a PSK cloud that slipped the C42 gate)
        return "qpsk"
    s3, s5, s9 = _spread(a, 3), _spread(a, 5), _spread(a, 9)
    if s3 * s9 / (s5 * s5 + 1e-18) > _D32:  # only a 5-ring cross collapses at k=5
        return "32qam"
    # amplitude-ring ratio, NEVER best-fit EVM (a denser grid always fits closer)
    return "16qam" if s3 / (s9 + 1e-12) < _R39 else "64qam"
```

```python
_SNR_CEIL = 40.0  # above this the moments cannot resolve the noise term


def snr_m2m4(z: np.ndarray, mod: str) -> float:
    """Blind symbol-SNR (dB) from the 2nd/4th moments, corrected for the
    constellation kurtosis. Saturates at _SNR_CEIL (noise below the moments'
    resolution); NaN when the moments leave the valid region entirely."""
    ka = np.mean(np.abs(ideal_points(mod)) ** 4)  # unit-power points => already /M2**2
    m2, m4 = np.mean(np.abs(z) ** 2), np.mean(np.abs(z) ** 4)
    disc = 2 * m2**2 - m4
    if disc <= 0:
        return float("nan")
    s = np.sqrt(disc / (2 - ka))
    n = m2 - s
    return min(float(10 * np.log10(s / n)), _SNR_CEIL) if n > 0 else _SNR_CEIL


def _nearest_dist(w: np.ndarray, pts: np.ndarray) -> np.ndarray:
    """Distance to nearest ideal point. Running min over the M points instead of a
    (..., M) broadcast temporary: identical values, far less memory, ~1.5-5x faster."""
    d = np.abs(w - pts[0])
    for p in pts[1:]:
        d = np.minimum(d, np.abs(w - p))
    return d


def align(z: np.ndarray, mod: str) -> np.ndarray:
    """AGC + resolve the M-fold rotation ambiguity by minimizing nearest-point distance."""
    z = _agc(z)
    pts = ideal_points(mod)
    sub = z[:1000]
    ang = np.linspace(0, 2 * np.pi / mod_symmetry(mod), 64, endpoint=False)
    rot = sub[None, :] * np.exp(1j * ang)[:, None]
    cost = _nearest_dist(rot, pts).mean(axis=1)
    i = int(np.argmin(cost))
    c0, c1, c2 = cost[(i - 1) % 64], cost[i], cost[(i + 1) % 64]  # parabolic sub-grid refine
    denom = c0 - 2 * c1 + c2
    # clip: on a flat cost surface (e.g. pure noise) the fit can explode
    delta = float(np.clip(0.5 * (c0 - c2) / denom, -1, 1)) if denom else 0.0
    return z * np.exp(1j * (ang[i] + delta * (ang[1] - ang[0])))


@dataclass
class Quality:
    evm: float
    mer_db: float
    lock: float
    occupied: int


def quality(z: np.ndarray, mod: str) -> Quality:
    """Alignment + EVM/MER, plus an occupancy-gated lock score in [0, 100]."""
    z = align(z, mod)
    pts = ideal_points(mod)
    m = pts.size
    idx = np.argmin(np.abs(z[:, None] - pts[None, :]), axis=1)
    d = pts[idx]
    evm = float(np.sqrt(np.mean(np.abs(z - d) ** 2) / np.mean(np.abs(d) ** 2)))
    mer_db = float(-20 * np.log10(evm + 1e-12))
    thr = max(3, 0.6 * len(z) / m)
```

```
    occupied = int(np.sum(np.bincount(idx, minlength=m) >= thr))
    lock = float(np.clip((mer_db - 6) / 19 * 100, 0, 100) * (occupied / m))
    return Quality(evm, mer_db, lock, occupied)
```

```
"""Optional numba acceleration for the sequential feedback loops (adaptive equalizer
taps, decision-directed carrier) that CANNOT be vectorized -- each update depends on
the previous output, so they are the only python-loop hot spots in an otherwise
vectorized pipeline. Numba is an OPTIONAL extra (`pip install signus[fast]`): when it
is absent `njit` degrades to a no-op and the identical bodies run as plain numpy, so
results are unchanged and the numpy+scipy-only baseline still works. The kernels use
the same expressions as the pure-numpy originals, so the numba path stays numerically
equivalent (verified: byte-identical `w @ win`, argmin/abs/angle exact)."""

import os

import numpy as np

HAVE_NUMBA = False
if not os.environ.get("SIGNUS_NO_NUMBA"):  # escape hatch: force the pure-numpy path
    try:
        import numba
        njit = numba.njit(cache=True, fastmath=False)  # fastmath off: keep IEEE results
        HAVE_NUMBA = True
    except ImportError:  # numba absent: same bodies run as plain numpy (slower, identical)
        pass
if not HAVE_NUMBA:
    def njit(fn):
        return fn


@njit
def cma_dd_equalize(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
    """Symbol-spaced CMA acquisition -> decision-directed LMS -> converged refilter."""
    n = x.size
    mid = n_taps // 2
    xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
                         np.zeros(mid, dtype=np.complex128)))
    w = np.zeros(n_taps, dtype=np.complex128)
    w[mid] = 1.0
    out = np.empty(n, dtype=np.complex128)
    warm = min(warmup, n)
    for i in range(warm):                        # Phase 1: CMA (blind, modulus-only)
        win = xp[i:i + n_taps]
        y = w @ win
        w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
    for i in range(warm, n):                     # Phase 2: decision-directed LMS
        win = xp[i:i + n_taps]
        y = w @ win
        d = pts[np.argmin(np.abs(y - pts))]
        w = w - mu_dd * (y - d) * np.conj(win)
    for i in range(n):                           # refilter whole record with final taps
        win = xp[i:i + n_taps]
        y = w @ win
        d = pts[np.argmin(np.abs(y - pts))]
        w = w - mu_dd * (y - d) * np.conj(win)
        out[i] = y
    return out


@njit
def cma_dd_equalize_fse(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
    """T/2 fractionally-spaced: 2 CMA passes -> 4 DD passes -> tracked output pass."""
    nsym = x.size // 2
    mid = n_taps // 2
```

```python
    xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
                         np.zeros(n_taps, dtype=np.complex128)))
    w = np.zeros(n_taps, dtype=np.complex128)
    w[mid] = 1.0
    out = np.empty(nsym, dtype=np.complex128)
    warm = min(warmup, nsym)
    for _ in range(2):                          # CMA acquisition, 2 data-reuse passes
        for i in range(warm):
            win = xp[2 * i:2 * i + n_taps]
            y = w @ win
            w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
    for _ in range(4):                          # decision-directed refinement
        for i in range(nsym):
            win = xp[2 * i:2 * i + n_taps]
            y = w @ win
            d = pts[np.argmin(np.abs(y - pts))]
            w = w - mu_dd * (y - d) * np.conj(win)
    for i in range(nsym):                       # final tracked output pass
        win = xp[2 * i:2 * i + n_taps]
        y = w @ win
        d = pts[np.argmin(np.abs(y - pts))]
        w = w - mu_dd * (y - d) * np.conj(win)
        out[i] = y
    return out


@njit
def dd_carrier(z, pts, alpha, beta):
    """Per-symbol decision-directed PI carrier loop (general PSK/QAM, not PSK Costas)."""
    out = np.empty(z.size, dtype=np.complex128)
    phase = 0.0
    freq = 0.0
    for i in range(z.size):
        y = z[i] * np.exp(-1j * phase)
        d = pts[np.argmin(np.abs(y - pts))]
        e = np.angle(y * np.conj(d))
        out[i] = y
        phase += freq + alpha * e
        freq += beta * e
    return out
```

```python
"""FSK/CPM receive path: constant-envelope gate + frequency-discriminator demod.
Runs on the DC-blocked analytic burst, before any carrier work (linear front-end
does not apply). Imports only constellation reference data (anti-shared-bug rule)."""

import numpy as np
from scipy.ndimage import uniform_filter1d
from scipy.signal import firwin, resample_poly, welch

from .constellations import fsk_levels, to_bits
from .dsp import _kmeans1d, _parab, analytic, mix

_CV_MAX = 0.24    # envelope coeff-of-variation ceiling. Recalibrated on a broad grid: real
# FSK (all h x SNR>=10 x high baud) tops out at cv 0.224, but constant-modulus PSK at very
# high baud (fs/baud~3) or near +-fs/2 dips to cv>=0.250 and used to slip the old 0.30 gate
# (false FSK). 0.24 sits in the 0.026 gap -> keeps every real FSK, rejects the PSK misfires.
_SEP_MIN = 3.1    # instantaneous-freq 2-means separation floor (FSK>=3.39, rest<=2.85)
_K4_RATIO = 2.6   # sp(k=2)/sp(k=4) collapse above which 4 levels beat 2
_MIN_SYMS = 8     # fewer symbols than this cannot cluster 2/4 levels (empty-quantile crash)
_PROM_MIN = 10.0  # a symbol-rate line peak/median below this is unreliable -- BUT only confident-
#                   wrong when paired with too few symbols (a long low-index burst has a weak line
#                   yet a correct baud). The sweep sits at prom 23-73.
_NSYM_MIN = 200   # ...so reject a weak line only below this symbol count. A 100 floor was tried
#                   (to recover more short bursts) but an independent grid found weak-prominence
#                   half-rate folds in the 100-199 band that still cluster tight enough to lock >=60
#                   (msk h=0.5 baud 24-32% off, lock 61-65) -- confident-wrong. 200 keeps them out;
#                   precision over recall (the short-burst confident-wrong is the cardinal sin).
_DECIM_FRAC = 0.30  # decimate a heavily-oversampled burst so the tones fill ~this of Nyquist
_DECIM_MIN = 3      # ...but only when that needs a factor >= this, so the sps~10 demod grid
#                     (fsk2/fsk4/msk baud=fs/10) is untouched -> the sweep stays byte-identical


def _bw99(x: np.ndarray, fs: float) -> float:
    """Two-sided 99%-power occupied bandwidth (Hz), noise-pedestal subtracted."""
    f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
    f, p = np.fft.fftshift(f), np.fft.fftshift(p)
    p = np.maximum(p - np.median(p), 0.0)
    cs = np.cumsum(p) / (p.sum() + 1e-30)
    lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]  # clamp BOTH edges: a degenerate PSD
    hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]  # (all-zero after floor subtraction)
    return float(abs(hi - lo))                           # otherwise indexes one past the end


def _dcblock(x: np.ndarray) -> np.ndarray:
    """Complex128 analytic burst, DC-blocked (front-end convention)."""
    x = analytic(x)
    return x - x.mean()


def _ifreq(x: np.ndarray, fs: float) -> np.ndarray:
    """Instantaneous frequency (Hz) from sample-to-sample phase differences."""
    return np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)


def _sep(a: np.ndarray) -> float:
    """2-means between/within separation ratio of a 1-D sample set."""
    c, _, sp = _kmeans1d(a, 2)
    return abs(c[1] - c[0]) / (sp + 1e-9)


def _est_baud(f: np.ndarray, fs: float) -> tuple[float, float]:
```

```
    """Symbol rate from the spectral line of |diff(smoothed f)| (transition impulses),
    harmonic-folded to the fundamental (2-level lines alias to 2*baud). Returns
    (baud, prominence) where prominence = peak/median of the in-band line: a weak line
    (few symbols, no clean symbol clock) means the baud is UNRELIABLE -> caller rejects."""
    d = np.abs(np.diff(uniform_filter1d(f, 2)))
    d = d - d.mean()
    n = 1 << int(np.ceil(np.log2(d.size)))
    spec = np.abs(np.fft.rfft(d * np.hanning(d.size), n))
    freqs = np.fft.rfftfreq(n, 1 / fs)
    df = freqs[1] - freqs[0]
    lo, hi = 0.002 * fs, 0.45 * fs
    idx = np.where((freqs >= lo) & (freqs <= hi))[0]
    prom = float(spec[idx].max() / (np.median(spec[idx]) + 1e-20)) if idx.size else 0.0
    k = idx[int(np.argmax(spec[idx]))]

    def _hp(kk: int) -> float:                       # harmonic-comb energy: the TRUE fundamental
        return float(sum(spec[m * kk] for m in range(1, 6) if m * kk < spec.size))  # captures most
    #   in-range harmonics ONLY: clamping out-of-range ones to the last bin multi-counted the
    #   Nyquist bin, inflating _hp at a 2*baud peak enough to veto the legitimate fold -> 2x baud.

    for div in (3, 2):
        ks = int(round(freqs[k] / div / df))
        if freqs[ks] >= lo:
            w = spec[max(0, ks - 2):ks + 3]
            kk = max(0, ks - 2) + int(np.argmax(w)) if w.size else k
            # fold to the sub-harmonic ONLY if it is genuinely the fundamental -- its harmonic comb
            # must beat the current peak's. A real baud/2 fundamental captures MORE harmonic lines
            # than the 2*baud peak; spurious low-h/low-SNR energy at baud/2 does not (it once
            # slipped the 0.34 gate and folded baud -> baud/2: a confident WRONG decode). The comb
            # guard vetoes that. On the sps~10 grid the fold never fired (0.34 unmet) -> sweep same.
            if w.size and w.max() > 0.34 * spec[k] and _hp(kk) >= _hp(k):
                k = kk
    return float(freqs[k] + _parab(20 * np.log10(spec + 1e-20), k) * df), prom


def fsk_gate(x: np.ndarray, fs: float) -> bool:
    """True when the burst looks like FSK/CPM: near-constant envelope AND a bimodal
    instantaneous frequency. Calibrated on generate() over all GEN_MODS x seeds 0..3
    x snr {25,15}: every linear mod fails one test with margin -- bpsk/dbpsk have
    cv~0.40 (>_CV_MAX), the rest have freq-sep<=2.85 (<_SEP_MIN); fsk2/fsk4/msk keep
    cv<=0.22 and freq-sep>=3.39 down to snr 10 (margins ~0.08 and ~0.29).
    Recenter by the mean instantaneous frequency first: at a large carrier offset a
    linear mod's phase-transition spikes wrap past +-fs/2 and fake a second mode."""
    x = _dcblock(x)
    a = np.abs(x)
    if a.std() / (a.mean() + 1e-12) >= _CV_MAX:
        return False
    f = _ifreq(x, fs)
    x = x * np.exp(-2j * np.pi * np.mean(f) / fs * np.arange(x.size))
    return bool(_sep(uniform_filter1d(_ifreq(x, fs), 4)) > _SEP_MIN)


def analyze_fsk(x: np.ndarray, fs: float) -> dict:
    """Full FSK/MSK demod by frequency discrimination. Returns mod, fc, baud, h,
    lock (0-100), level_freqs (Hz, sorted, centered), symbols (normalized f per
    symbol) and Gray-demapped bits."""
    x = _dcblock(x)
    f = _ifreq(x, fs)
    # A carrier OFFSET (tones sit at fc +- dev, not around DC) corrupts the symbol-rate estimate two
    # ways -- off-centre it trips _est_baud's harmonic fold, and the tones fall outside the decim
```

```python
    # LPF passband -- either way a confident WRONG baud. Recentre the discriminator on the carrier
    # (mean instantaneous freq) first; only for a significant offset, so a centred burst
    # (fc~0, the whole demod/sweep grid) is byte-identical. Levels are already centred downstream.
    fc0 = float(np.mean(f))
    if abs(fc0) > 0.005 * fs:
        x = mix(x, fs, fc0)
        f = _ifreq(x, fs)
    else:
        fc0 = 0.0
    baud, prom = _est_baud(f, fs)
    # heavy oversampling (fs/baud >> 10) buries the symbol-rate line under broadband transition
    # artifacts -> _est_baud locks a spurious peak (baud ~25% off) while the clean tones still
    # cluster (lock ~94): a confident WRONG decode. Re-estimate baud on a decimated copy (tones ->
    # ~_DECIM_FRAC of Nyquist, as the survey channelizer does). Gated at _DECIM_MIN so the sps~10
    # grid is untouched (fsk2/fsk4/msk baud=fs/10 -> d<3 -> no re-estimate -> sweep byte-identical).
    bw = _bw99(x, fs)
    d = int(_DECIM_FRAC * fs / bw) if bw > 0 else 1
    if d >= _DECIM_MIN:
        xd = np.convolve(x, firwin(129, min(0.9 * (fs / d) / 2, bw) / (fs / 2)), "same")
        baud, prom = _est_baud(_ifreq(resample_poly(xd, 1, d), fs / d), fs / d)
    if baud <= 0:
        raise ValueError("FSK 버스트에서 심볼율을 추정할 수 없습니다 (너무 짧음)")
    sps = fs / baud
    f = uniform_filter1d(f, max(1, int(round(sps / 2))))
    resid = float(f.mean())
    fc = fc0 + resid                    # absolute carrier = recentre offset + in-band residual
    fcen = f - resid
    nsym = int(fcen.size / sps) - 1
    # a WEAK symbol-rate line (prom < _PROM_MIN) is unreliable ONLY when symbols are also too few to
    # average (nsym < _NSYM_MIN): a short burst's spurious peak still clusters tight (few points ->
    # high lock) = a confident WRONG baud. A long low-index (h~0.35) burst also has a weak line but
    # MANY symbols -> its baud is correct -> NOT rejected. Sweep: nsym~3000/prom 23-73 -> untouched.
    if nsym < _MIN_SYMS or (prom < _PROM_MIN and nsym < _NSYM_MIN):
        raise ValueError(f"FSK 버스트가 너무 짧거나 심볼율이 불확실합니다 ({nsym} 심볼)")
    ctr = (np.arange(nsym) + 0.5) * sps

    # symbol sampling phase: the offset whose sampled levels separate best
    v, best = fcen[:nsym], -1.0
    for o in np.linspace(0, sps, 8, endpoint=False):
        s = fcen[np.clip((ctr + o).astype(int), 0, fcen.size - 1)]
        sep = _sep(s)
        if sep > best:
            best, v = sep, s

    # level count via spread-collapse (k=2 vs k=4), then cluster
    k = 4 if _kmeans1d(v, 2)[2] / (_kmeans1d(v, 4)[2] + 1e-9) > _K4_RATIO else 2
    c, lab, spread = _kmeans1d(v, k)
    order = np.argsort(c)
    ranks = np.argsort(order)[lab]          # per-symbol level index, 0 = lowest freq
    csort = c[order]
    mid = float(csort.mean())               # debias center: symmetric levels sum ~0
    fc, csort, v = fc + mid, csort - mid, v - mid
    spacing = float(np.mean(np.diff(csort)))
    h = spacing / baud

    mod = "msk" if (k == 2 and abs(h - 0.5) < 0.15) else f"fsk{k}"
    _, glabels = fsk_levels(mod)
    bits = to_bits(glabels[ranks], mod).astype(np.uint8)
    lock = float(np.clip((spacing / (spread + 1e-9) - 1.5) / 6.5 * 100, 0, 100))
    return dict(mod=mod, fc=fc, baud=baud, h=h, lock=lock, level_freqs=csort,
```

```
            symbols=v / (spacing / 2), bits=bits)
```

```python
"""Blind linear-chirp / CSS (LoRa) detection + characterization. A constant-envelope
signal whose instantaneous frequency ramps linearly de-chirps to a tone: the delay-
conjugate x[t]*conj(x[t-tau]) is a beat tone at mu*tau (NONZERO), unlike FSK/CW (beat at
DC), PSK/QAM (not constant-envelope), or FM-voice/noise (broadband). Characterize-only --
reported like analog/tone, NEVER demodulated: blind CSS symbol decode needs preamble
CFO/STO sync plus LoRa's reverse-engineered Gray/interleave/Hamming/whitening framing.
Calibrated on gen chirps vs every other family: chirp beat-PAR>=936, non-chirp<=264."""

import numpy as np
from scipy.ndimage import uniform_filter1d
from scipy.signal import welch

_PAR_MIN = 400.0   # delay-conjugate beat-tone peak-to-mean floor (chirp>=936, non-chirp<=264)


def _bw99(x: np.ndarray, fs: float) -> float:
    """99%-power occupied bandwidth (a chirp fills its band ~flat). Floor-subtracted so the
    noise pedestal in the channel's LPF passband does not inflate the width."""
    f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
    f, p = np.fft.fftshift(f), np.fft.fftshift(p)
    p = np.maximum(p - np.median(p), 0.0)    # remove the noise pedestal
    cs = np.cumsum(p) / (p.sum() + 1e-30)
    lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]  # clamp BOTH edges: a degenerate PSD
    hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]  # (all-zero after floor subtraction)
    return float(abs(hi - lo))                           # otherwise indexes one past the end


def _beat(x: np.ndarray, fs: float) -> tuple[float, float]:
    """(peak-to-mean, mu[Hz/s]) of the DC-nulled delay-conjugate spectrum."""
    x = x - x.mean()
    tau = max(1, x.size // 200)
    y = x[tau:] * np.conj(x[:-tau])
    nf = 1 << int(np.ceil(np.log2(y.size)))
    spec = np.abs(np.fft.fftshift(np.fft.fft((y - y.mean()) * np.hanning(y.size), nf))) ** 2
    fr = np.fft.fftshift(np.fft.fftfreq(nf, 1 / fs))
    spec[np.abs(fr) < 0.002 * fs] = 0.0    # null DC band: FSK/CW/tone beat here, chirps do not
    k = int(np.argmax(spec))
    return float(spec[k] / (spec.mean() + 1e-30)), float(fr[k] * fs / tau)


def is_chirp(x: np.ndarray, fs: float) -> bool:
    """True when a channel is a linear chirp / CSS (LoRa/FMCW). The beat-tone PAR is the
    discriminator: a huge margin (chirp>=936 vs PSK/QAM<=94, FSK<=91, FM<=264, noise<=13)
    makes an envelope-CV pre-gate redundant AND harmful (it rejects noisy-but-clear chirps)."""
    if x.size < 256:
        return False
    return _beat(x, fs)[0] >= _PAR_MIN


_SWEEP_PM_MAX = 1.45   # instantaneous-freq histogram peak-to-mean: a band-sweep stays ~1
_SWEEP_OCC_MIN = 0.6   # ...and fills its band; FSK sits on 2-4 discrete tones (peaky, sparse)


def sweeps_band(x: np.ndarray, fs: float, bins: int = 40) -> bool:
    """True when the instantaneous frequency SWEEPS the channel (linear chirp / CSS) rather
    than hopping between a few discrete FSK tones. A linear FMCW ramp reads bimodal to
    fsk_gate AND trips is_chirp's beat test, so triage needs this to tell a genuine band-sweep
    from FSK: a sweep's IF histogram is flat and fully occupied; FSK's spikes at its tones.
    Calibrated on extracted channels: 0/222 FSK/MSK (snr 8-40) pass, 214/216 FMCW + 60/60 LoRa
    pass (misses only the narrowest, gentlest ramps -- which stay 'fsk', never a regression).
```

```
    BOTH the raw and a lightly-smoothed IF must look like a sweep: at moderate SNR (~14) noise
    flattens integer-h FSK's raw histogram to sweep-like pm ~1.44 (knife-edge), but smoothing
    restores its tone spikes (pm 2.5+) while a genuine ramp stays flat (pm ~1.1) -- and at LOW
    SNR smoothing can flatten FSK instead, which the raw view still catches. The conjunction
    only ever narrows 'sweep', so every calibrated chirp above keeps its label (margin >=0.3)."""
    if x.size < 256:
        return False
    x = x - x.mean()
    f0 = np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)
    for f in (f0, uniform_filter1d(f0, 4)):
        lo, hi = np.percentile(f, 1), np.percentile(f, 99)
        if hi - lo < 1:
            return False
        fv = f[(f >= lo) & (f <= hi)]
        h = np.histogram(fv, bins=bins)[0] / fv.size
        pm = h.max() / (h.mean() + 1e-30)           # discrete tones spike this; a sweep ~1
        occ = float((h > 0.005).mean())             # a sweep fills the band; tones occupy few bins
        if not (pm < _SWEEP_PM_MAX and occ >= _SWEEP_OCC_MIN):
            return False
    return True


def analyze_chirp(x: np.ndarray, fs: float) -> dict:
    """Characterize a chirp: {mu[Hz/s], up, bw, par, sf, rs, tsym}. A LoRa hypothesis
    (sf, symbol rate, symbol time) is filled when bw^2/|mu| snaps to 2^(7..12); otherwise
    it stays a generic linear chirp / FMCW. bw is measured on the channel, not asserted."""
    par, mu = _beat(x, fs)
    bw = _bw99(x, fs)
    sf = rs = tsym = None
    if mu != 0 and bw > 0:
        r = float(np.log2(bw * bw / abs(mu)))
        if 7 <= round(r) <= 12 and abs(r - round(r)) < 0.3:    # power-of-two chip count -> LoRa
            sf = int(round(r))
            rs = abs(mu) / bw
            tsym = 2 ** sf / bw
    return {"mu": mu, "up": bool(mu > 0), "bw": float(bw),
            "par": par, "sf": sf, "rs": rs, "tsym": tsym}
```

```python
"""End-to-end blind demodulation. Two families:
  linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
                    classify -> fine sync -> [equalizer rescue] -> quality
  fsk (CPFSK/MSK):  frequency discriminator (gated by constant envelope)
Classification runs BEFORE fine sync so the decision-directed loop knows the
constellation. Differential demapping is an option, not a detection: dqpsk and
qpsk share a constellation."""

import json
from dataclasses import dataclass, field

import numpy as np

from . import classify as cl
from . import dsp, triage
from .channelize import extract
from .chirp import analyze_chirp, is_chirp, sweeps_band
from .constellations import demap_bits, demap_diff_bits, mod_order
from .detect import Detection, detect
from .eq import equalize, equalize_fse
from .fsk import analyze_fsk, fsk_gate
from .sigio import Meta, parse_name, read
from .spectrum import spectrum, waterfall
from .sync import find_preamble

_BITS_CAP = 65536
_EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
_EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
_EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
_EQ_QAM_FLOOR = 60.0  # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop can
#                       drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51), a
#                       confident WRONG order. Genuine multipath recoveries clear ~72; the sweep's
#                       eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identical.
_BAUD_GAIN = 5.0   # an in-band baud hypothesis must beat the global one by this lock
_SHORT_LOCK = 60.0  # below this, retry the baud line with fewer blocks (short-burst rescue)
_SYNC_LOCK = 60.0   # below this, try a repeated-preamble carrier/timing sync (packetized bursts)
_ALIAS_MARGIN = 10.0  # a preamble carrier alias must beat the next by this lock, else ambiguous.
#                      This is the LOAD-BEARING rotation guard: even when the coarse anchor itself
#                      aliases and validates a rotation at ~80 lock, that rotation wins by a thin
#                      margin (~8) -- so a 10-lock margin refuses it. Because the margin catches it,
#                      the lock floor can stay modest (78, not 82) -> ~5% more genuine recoveries.
_SYNC_ACCEPT = 78.0  # ...AND clear this absolute lock: below it correct/rotated aliases overlap. A
#                      65 floor was tried (to recover more moderate bursts) but an independent
#                      snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-78 band
#                      (carrier off by one baud/L step -> confident garbage bits). Even 78 leaks a
#                      a few hard-corner rotations (a separate pre-existing gate limit): hold at 78.
_SYNC_ADIST = 1.5    # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
#                      soft backstop to the margin: it catches the one rotation the margin misses,
#                      but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
#                      large-offset recoveries. The three together give 0 confident-wrong across
#                      1920 hard-corner bursts at the max recall this gate reaches.
_DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"}  # pi4dqpsk arrives as qpsk -> dqpsk


@dataclass
class Result:
    meta: Meta
    family: str                    # 'linear' | 'fsk'
    burst: tuple[int, int]
    fc: float
```

```python
    baud: float
    mod: str
    lock: float
    symbols: np.ndarray            # aligned symbols (linear) / normalized levels (fsk)
    bits: np.ndarray
    iq_corr: np.ndarray            # exportable corrected baseband
    burst_x: np.ndarray = field(repr=False, default=None)  # for spectrum views
    symmetry: int = 0
    carrier_ambiguous: bool = False
    baud_conf: float = 0.0
    rolloff: float | None = None
    h: float | None = None         # FSK modulation index
    evm: float = float("nan")
    mer_db: float = float("nan")
    occupied: int = 0
    snr_est: float = float("nan")
    eq_applied: bool = False
    eq_mode: str | None = None     # 'sym' | 'fse'
    alias_resolved: bool = False
    baud_fallback: bool = False
    bursts: list = field(default_factory=list)
    burst_idx: int = 0
    preamble: object = None        # sync.Preamble when a repeated preamble drove the decode

    def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
        z = np.nan_to_num(self.symbols)   # a dead/DC capture -> NaN symbols -> invalid JSON
        if z.size > max_points:  # keep TIME ORDER: the UI animates this sequence
            z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
        det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
               "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
               "baud_conf": round(self.baud_conf, 1),
               "alias_resolved": self.alias_resolved,
               "baud_fallback": self.baud_fallback}
        det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
        rf = self.meta.rf_center                      # real RF = capture centre + baseband fc
        det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
        if self.h is not None:
            det["h"] = round(self.h, 3)
        if self.preamble is not None:                 # a repeated preamble drove the sync
            p = self.preamble
            det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
                               "conf": round(p.conf, 2), "start": p.start, "end": p.end}
        doc = {
            "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,  # dtype 은 한 줄
            "rf_center": self.meta.rf_center,          # 보고에 필수: lock 0 의 1순위 용의자다

            "family": self.family, "n_samples": int(self.iq_corr.size),
            "burst": {"start": self.burst[0], "end": self.burst[1]},
            "bursts": [{"start": s, "end": e} for s, e in self.bursts],
            "burst_idx": self.burst_idx,
            "detected": det,
            "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
                        "evm": _r(self.evm, 4)},
            "snr_est_db": _r(self.snr_est),
            "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
            "constellation": {"i": np.round(z.real, 4).tolist(),
                              "q": np.round(z.imag, 4).tolist()},
            "bits": "".join(map(str, self.bits[:_BITS_CAP])),
        }
        if views and self.burst_x is not None:
```

```python
            doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
            doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
        return doc

    def save_iq(self, path: str) -> None:
        np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)

    def save_symbols(self, path: str) -> None:
        np.save(path, self.symbols.astype(np.complex64))

    def save_bits(self, path: str, packed: bool = False) -> None:
        if packed:
            np.packbits(self.bits).tofile(path)
        else:
            with open(path, "w") as fh:
                fh.write("".join(map(str, self.bits)))

    def save_report(self, path: str) -> None:
        with open(path, "w") as fh:
            json.dump(self.to_json(), fh, indent=1)


def _r(v: float, nd: int = 2) -> float | None:
    return None if v is None or not np.isfinite(v) else round(float(v), nd)


def _fine(z: np.ndarray, mod: str) -> np.ndarray:
    """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
    return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)


def _blocks(n: int) -> int:
    """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
    low SNR) wants fewer, longer segments; only long records need more for
    stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
    return int(np.clip(n // 30000, 1, 8))


def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
    """Matched filter + timing + classify + fine sync at one baud hypothesis."""
    alpha = dsp.est_rolloff(xd, fs, baud)
    ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
    raw = dsp.timing(ym, 4)
    mod = cl.classify(raw, symmetry)
    # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
    # the bulk phase first kills the loop transient that would corrupt the first symbols
    syms = _fine(raw, mod)
    return alpha, ym, raw, mod, syms, cl.quality(syms, mod)


def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
    """Equalize with a seed constellation, then let the OPENED eye pick the mod:
    heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
    with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
    z1 = eq_fn(z, seed_mod)
    cands = []
    for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}:   # symmetry too, per the docstring
        s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
        cands.append((cl.quality(s, m), s, m))
    best = max(cands, key=lambda c: c[0].lock)
```

```python
    # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
    # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
    # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
    # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
    # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
    lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
    if lo and best[2] == "qpsk":
        return lo[0]
    return best


def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
            burst: int | None = None) -> Result:
    """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
    `burst` selects one detected burst (default: the most energetic)."""
    if x.size:
        # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
        # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
        # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
        # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
        # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
        # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
        scale = float(np.percentile(np.abs(x[::16]), 99))  # strided: scale detection needs the
        #             BULK magnitude, not every sample -- 16x cheaper on a large direct capture
        if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
            x = x / scale
    x = dsp.analytic(x)
    x = x - x.mean()  # DC block: LO leakage otherwise corrupts every estimator
    if x.size < 64:   # empty / trivially short: nothing to estimate, and it crashes downstream
        raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
    rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
    if rms and (rms < 1e-6 or rms > 1e6):  # only PATHOLOGICAL scale: normalize so the scale-variant
        x = x / rms                        # spots (fsk_gate's CV epsilon, est_carrier's x**p over/
    #                                        underflow) stay invariant. Normal captures rms~O(1) ->
    #                                        untouched -> the acceptance sweep is byte-identical.
    bursts = dsp.find_bursts(x, meta.fs)
    if burst is None or not 0 <= burst < len(bursts):
        burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
    s, e = bursts[burst]
    xb = x[s:e] if e - s >= 64 else x

    # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
    # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
    # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
    # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
    if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
        info = analyze_chirp(xb, meta.fs)
        kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
        raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) — "
                         "성상도 복조 대상이 아닙니다 (서베이 모드에서 특성 보고)")

    if fsk_gate(xb, meta.fs):
        r = analyze_fsk(xb, meta.fs)
        sym = np.asarray(r["symbols"], dtype=np.complex128)
        return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
                      sym, r["bits"], xb, burst_x=xb, h=r["h"],
                      bursts=bursts, burst_idx=burst)

    fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
    fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
```

```
    amb = abs(symmetry * fc) > 0.4 * meta.fs
    xd = dsp.mix(xb, meta.fs, fc)

    baud, conf = dsp.est_baud(xd, meta.fs)
    fell = False
    alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
    bw = dsp.occupied_bw(xd, meta.fs)
    if bw and not (0.72 * bw <= baud <= 1.05 * bw):
        # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
        # (and under harsh channels) data junk outruns the true line. Demod the
        # in-band hypotheses too and let lock decide, with a clear margin.
        for lo in (0.80, 0.67):
            b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
            if abs(b2 - baud) <= 0.01 * b2:
                continue
            t = _demod(xd, meta.fs, b2, symmetry)
            if t[5].lock > q.lock + _BAUD_GAIN:
                alpha, ym, raw, mod, syms, q = t
                baud, conf, fell = b2, c2, True

    if q.lock < _SHORT_LOCK:
        # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
        # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
        # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
        # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
        for nb in (2, 1):
            b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
            if abs(b2 - baud) <= 0.01 * b2:
                continue
            t = _demod(xd, meta.fs, b2, symmetry)
            if t[5].lock > q.lock + _BAUD_GAIN:
                alpha, ym, raw, mod, syms, q = t
                baud, conf, fell = b2, c2, True

    eq_applied, eq_mode, pre = False, None, None
    if q.lock < _EQ_LOCK:  # ISI (multipath) is what a phase-only loop cannot fix
        # CMA is modulus-only, so it opens the eye without knowing the constellation.
        qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
        mode = "sym"
        if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
            # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
            # the clock-tracked 2 sps stream (unaliased spectrum, longer inverse)
            q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
                                 dsp.timing(ym, 4, out=2), mod, symmetry)
            if q2.lock > qe.lock:
                qe, se, me, mode = q2, s2, m2, "fse"
        floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
        if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
            syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
    elif mod in ("16qam", "32qam", "64qam"):
        # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
        # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
        # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
        # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched.
        qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
        mode = "sym"
        if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
            # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
            # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
            # QAM (no lower order) never pays for the FSE.
```

```
            qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
                                 dsp.timing(ym, 4, out=2), mod, symmetry)
            mode = "fse"
        if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
            syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode

    if q.lock < _SYNC_LOCK:
        # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
        # carrier/baud/timing from only a few symbols, where the blind M-th-power estimator (needing
        # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock, so
        # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
        # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -- a
        # random / no-preamble signal yields no plateau or a CFO that does not improve lock and is
        # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the baud
        # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-fold
        # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
        ps = find_preamble(xb, meta.fs, baud_hint=baud)
        if ps is not None:
            # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by baud/L
            # rotates a whole constellation position per symbol, so the eye still looks locked but
            # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coarse
            # M-th-power fc lands within ~1 alias step of the truth even on these short bursts, so
            # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct alias
            # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
            step = meta.fs / ps.period
            k0 = round((fc - ps.cfo_hz) / step)
            cands = []
            for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
                for k in range(k0 - 3, k0 + 4):
                    cfo = ps.cfo_hz + k * step
                    data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
                    if data.size < 256:
                        continue
                    for sm in (2, 4, 8):
                        t = _demod(data, meta.fs, float(b2), sm)
                        cands.append((t[5].lock, cfo, float(b2), sm, t))
            cands.sort(key=lambda c: -c[0])
            # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG one by a
            # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear gap;
            # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rather
            # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete.)
            # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -- a
            # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alone
            # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38). Two
            # conjunctive conditions favour precision over recall: (1) a strong absolute lock, and
            # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -- a
            # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
            # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); on the
            # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
            if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
                top = cands[0]
                other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
                anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
                if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
                    alpha, ym, raw, mod, syms, q = top[4]
                    fc, baud, symmetry = top[1], top[2], top[3]
                    eq_applied, eq_mode, pre = False, None, ps
                    # diagnostics must describe the ACCEPTED decode, not the abandoned blind
                    # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
                    # keeps alias_resolved False), the baud from the preamble period (no
```

```
                # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
                fc0, conf, fell = fc, 0.0, False
                amb = abs(symmetry * fc) > 0.4 * meta.fs

    bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
        else demap_bits(syms, mod)
    return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
                  burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
                  rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
                  snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
                  alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
                  bursts=bursts, burst_idx=burst, preamble=pre)


def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
                 dtype: str | None = None, endian: str | None = None,
                 bitrev: bool | None = None, diff: bool = False,
                 burst: int | None = None, rf: float | None = None) -> Result:
    """Analyze a file; explicit args override SigMF/filename tokens."""
    from .sigio import parse_sigmf
    m = parse_sigmf(path) or parse_name(path)
    meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
                endian or m.endian, m.bitrev if bitrev is None else bitrev,
                rf_center=rf if rf is not None else m.rf_center)
    x, meta = read(path, meta)
    return analyze(x, meta, diff, burst)


# --- wideband survey: many emitters in one capture --------------------------

@dataclass
class Emitter:
    detection: Detection
    kind: str                       # linear|fsk|chirp|analog|tone|tooshort|error
    abs_fc: float                   # emitter carrier vs capture centre (Hz)
    result: Result | None = None    # demod result for digital kinds
    info: dict | None = None        # characterization for non-digital kinds (e.g. chirp params)

    def to_json(self) -> dict:
        d = self.detection
        doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
               "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
                       "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
                       "baud_hint": round(d.baud_hint, 1)}}
        if self.info is not None:
            doc["info"] = self.info
        if self.result is not None:
            r = self.result
            doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
                       mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
        return doc

    def to_detail(self, rf_center: float | None = None) -> dict:
        """Web drill-down payload: the box summary plus, for a digital emitter, the
        full analyze result so the UI reuses its single-signal detail view verbatim.
        The channel is demodulated at baseband (no rf_center), and r.fc is only the
        residual within-channel offset -- so the emitter's absolute RF is the capture
        centre plus abs_fc (the full offset from capture centre), injected here."""
        doc = self.to_json()
        if self.result is not None:
```

```python
            doc["result"] = self.result.to_json()
            if rf_center is not None:
                doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
        return doc


@dataclass
class Survey:
    meta: Meta
    emitters: list[Emitter] = field(default_factory=list)

    def to_json(self) -> dict:
        return {"fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,
                "n_emitters": len(self.emitters),
                "emitters": [e.to_json() for e in self.emitters]}


def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
    """Detect every emitter in a wideband capture, extract each to its own baseband
    channel, and demodulate the digital ones with the unchanged single-signal chain.
    A capture holding one signal that fills the band collapses to a single emitter."""
    xa = dsp.analytic(x)
    xa = xa - xa.mean()
    emitters = []
    for d in detect(xa, meta.fs, **detect_kw):
        ch, fs_ch = extract(xa, meta.fs, d)
        if ch.size < 256:
            emitters.append(Emitter(d, "tooshort", d.fc))
            continue
        kind = triage.family(ch, fs_ch)
        if kind in ("linear", "fsk"):
            # triage only gates digital vs not; analyze's own family is authoritative.
            # Isolate each channel: a degenerate one must not abort the whole survey.
            try:
                r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
                emitters.append(Emitter(d, r.family, d.fc + r.fc, r))
            except Exception:
                emitters.append(Emitter(d, "error", d.fc))
        else:
            info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
            emitters.append(Emitter(d, kind, d.fc, info=info))
    return Survey(meta, emitters)


def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
    """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
    is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
    result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
    capture overview waterfall and every emitter's drill-down detail. Additive: does
    not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
    xa = dsp.analytic(x)
    xa = xa - xa.mean()
    if len(detect(xa, meta.fs)) <= 1:
        r = analyze(x, meta, diff=diff)
        out = {"mode": "single", "result": r.to_json()}
        if len(r.bursts) > 1:   # multi-burst signal -> whole-record map for burst selection
            out["overview"] = {"n": int(xa.size), "fs": meta.fs,
                               "waterfall": waterfall(xa, meta.fs)}
        return out
    sv = survey(x, meta, diff=diff)
```

```python
    return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
            "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),
                         "waterfall": waterfall(xa, meta.fs)},
            "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}


def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
                dtype: str | None = None, endian: str | None = None,
                bitrev: bool | None = None, diff: bool = False,
                rf: float | None = None) -> Survey:
    """Survey a capture file; explicit args override SigMF/filename tokens."""
    from .sigio import parse_sigmf
    m = parse_sigmf(path) or parse_name(path)
    meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
                endian or m.endian, m.bitrev if bitrev is None else bitrev,
                rf_center=rf if rf is not None else m.rf_center)
    x, meta = read(path, meta)
    return survey(x, meta, diff=diff)
```

```python
"""CLI: analyze FILE / survey / serve. 합성·채점(gen/dataset/sweep)은 개발기 전용 lab.py 가
단다 — 격리망 장비에는 그 파일이 없어 명령 자체가 없다."""

import argparse
import importlib.util
import re
import sys
from zlib import crc32

from .pipeline import analyze_file, survey_file
from .sigio import sidecar_read

_DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")


# --- 손으로 옮기는 한 줄 ---------------------------------------------------------
# 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
# 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
# 본말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.

_ALPHA = "0123456789abcdefghjkmnpqrstvwxyz"  # 0/o, 1/l/i 처럼 화면에서 헷갈리는 글자를 뺀 32자
_BRIEF_FLAGS = [("eq", lambda d: d["eq"]["applied"]),
                ("al", lambda d: d["detected"]["alias_resolved"]),
                ("fb", lambda d: d["detected"]["baud_fallback"]),
                ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
                ("pre", lambda d: "preamble" in d["detected"])]


def check_code(text: str) -> str:
    """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
    않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam'(샘플레이트 10배!) 이 같은
    코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
    body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
    n = crc32(body.encode())
    return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))


def brief(doc: dict, mode: str) -> str:
    """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 딕셔너리에서 만든다 -- 객체
    속성을 직접 포맷하면 to_json 이 이미 한 반올림과 두 번 겹쳐 장비와 맥이 갈린다."""
    head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}-{doc['dtype']}"
    if mode == "sv":
        lines = [f"{head} n{doc['n_emitters']}"]
        for i, e in enumerate(doc["emitters"][:12]):
            baud = f" bd{e['baud']:.0f}" if e.get("baud") else ""
            lock = f" lk{e['lock']:.0f}" if e.get("lock") is not None else ""
            lines.append(f"{i} fc{e['abs_fc']:.0f}{baud}{lock} {e.get('mod') or e['kind']}")
        if len(doc["emitters"]) > 12:
            lines.append(f"...{len(doc['emitters']) - 12}개 생략")
    else:
        d, q = doc["detected"], doc["quality"]
        p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
        if q["mer_db"] is not None:
            p.append(f"mer{q['mer_db']:.1f}")
        if d.get("h") is not None:                  # FSK 는 롤오프 대신 변조지수
            p.append(f"h{d['h']:.2f}")
        elif d["rolloff"] is not None:
            p.append(f"rl{d['rolloff']:.2f}")
        if len(doc["bursts"]) > 1:
            p.append(f"b{doc['burst_idx'] + 1}/{len(doc['bursts'])}")
```

```python
        p += [f for f, get in _BRIEF_FLAGS if get(doc)]
        lines = [" ".join(p)]
    body = "\n".join(lines)
    return f"{body} #{check_code(body)}"


# --- analyze / survey / serve ----------------------------------------------------

def _analyze(args: argparse.Namespace) -> int:
    r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
                     "be" if args.be else None, args.bitrev or None, args.diff,
                     None if args.burst is None else args.burst - 1, rf=args.rf)
    j = r.to_json(views=False)          # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
    d = j["detected"]
    truth = (sidecar_read(args.file) or {}).get("truth")  # display only, never detection
    if len(r.bursts) > 1:
        print(f"버스트 {len(r.bursts)}개 감지 — {r.burst_idx + 1}번째 분석 (--burst N 으로 선택)")
    print(f"모드      {d['mod']}" + (f"   (정답 {truth['mod']})" if truth else ""))
    print(f"반송파    {d['fc']:.1f} Hz" + (f"   (정답 {truth['fc']:.1f})" if truth else ""))
    if d["rf_hz"] is not None:
        print(f"실제 RF   {d['rf_hz'] / 1e6:.6f} MHz")
    print(f"심볼레이트 {d['baud']:.1f} Hz" + (f"   (정답 {truth['baud']:.1f})" if truth else ""))
    fsk = r.family == "fsk"
    tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프    {d['rolloff']:.2f}"
    mer = "" if fsk else f"   MER {r.mer_db:.1f} dB"
    print(f"{tail}   lock {r.lock:.1f}{mer}")
    if r.eq_applied:
        print("등화기 적용: 다중경로 ISI 보정됨"
              + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
    if r.alias_resolved:
        print("반송파 앨리어스 보정: 스펙트럼 중심으로 후보 선택")
    if r.baud_fallback:
        print("심볼레이트 폴백: 점유대역폭 사전정보로 재탐색")
    if r.carrier_ambiguous:
        print("경고: 반송파 앨리어싱 가능 (|sym*fc| > 0.4*fs)")
    if args.save_iq:
        r.save_iq(args.save_iq)
    if args.save_symbols:
        r.save_symbols(args.save_symbols)
    if args.save_bits:
        r.save_bits(args.save_bits, args.packed)
    if args.report:
        r.save_report(args.report)
    if args.brief:                      # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 —
        print(brief(j, "an"))   # 조기 return 이면 위 저장들이 조용히 무시된다
    return 0


def _fhz(v: float) -> str:
    a = abs(v)
    if a >= 1e6:
        return f"{v / 1e6:+.3f} MHz"
    if a >= 1e3:
        return f"{v / 1e3:+.2f} kHz"
    return f"{v:+.0f} Hz"


def _survey(args: argparse.Namespace) -> int:
    """Wideband survey: detect every emitter, demodulate the digital ones."""
    s = survey_file(args.file, args.fs, args.fmt, args.dtype,
```

```python
                    "be" if args.be else None, args.bitrev or None, args.diff, rf=args.rf)
    rf0 = s.meta.rf_center
    note = f" · RF 중심 {rf0 / 1e6:.3f} MHz" if rf0 is not None else ""
    print(f"{len(s.emitters)}개 신호 감지 (샘플레이트 {s.meta.fs:.0f} Hz{note})")
    print(f"{'#':>2} {('실제 RF' if rf0 is not None else '중심주파수'):>12} {'대역폭':>10}"
          f" {'분류':>7} {'변조':>7} {'심볼레이트':>11} {'lock':>5}")
    for i, e in enumerate(s.emitters):
        r = e.result
        mod = r.mod if r else "—"
        baud = f"{r.baud:.0f}" if r else "—"
        lock = f"{r.lock:.0f}" if r else "—"
        kind = {"linear": "디지털", "fsk": "FSK", "analog": "아날로그",
                "tone": "순수톤", "tooshort": "너무짧음", "error": "오류"}.get(e.kind, e.kind)
        fc = e.abs_fc if rf0 is None else rf0 + e.abs_fc
        print(f"{i:>2} {_fhz(fc):>12} {_fhz(e.detection.bw):>10} {kind:>7}"
              f" {mod:>7} {baud:>11} {lock:>5}")
    if args.report:
        import json
        with open(args.report, "w") as fh:
            json.dump(s.to_json(), fh, indent=1)
    if args.brief:
        print(brief(s.to_json(), "sv"))
    return 0


def _read_args(p: argparse.ArgumentParser) -> None:
    """Read-side sample-format overrides, shared by analyze/survey (sidecar/filename win)."""
    p.add_argument("--fs", type=float)
    p.add_argument("--fmt", choices=("iq", "real"))
    p.add_argument("--dtype", choices=_DTYPE_CHOICES)
    p.add_argument("--be", action="store_true", help="big-endian 샘플")
    p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
    p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) — 실제 주파수로 보고")
    p.add_argument("--brief", action="store_true",
                   help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")


def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(prog="signus")
    sub = ap.add_subparsers(dest="cmd", required=True)

    a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
    a.add_argument("file")
    _read_args(a)
    a.add_argument("--save-iq")
    a.add_argument("--save-symbols")
    a.add_argument("--save-bits")
    a.add_argument("--packed", action="store_true")
    a.add_argument("--report")
    a.add_argument("--diff", action="store_true",
                   help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
    a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")

    sv = sub.add_parser("survey", help="광대역 캡처의 모든 신호 탐지 + 복조")
    sv.add_argument("file")
    _read_args(sv)
    sv.add_argument("--diff", action="store_true", help="차동 디맵")
    sv.add_argument("--report", help="JSON 리포트 저장 경로")

    s = sub.add_parser("serve", help="웹 UI 서버")
```

```python
    s.add_argument("--host", default="127.0.0.1")
    s.add_argument("--port", type=int, default=8000)

    # 합성 생성·채점(gen/dataset/sweep)은 개발기 전용이다. 격리망 장비에는 lab.py 를
    # 옮기지 않으므로 그 장비에서는 이 명령들이 아예 존재하지 않는다. try/except 로 삼키면
    # 개발기에서 lab 내부의 진짜 임포트 오류까지 "명령 없음"으로 둔갑하니 파일 유무만 본다.
    if importlib.util.find_spec("signus.lab"):
        from . import lab
        lab.add_commands(sub)

    args = ap.parse_args(argv)
    if args.cmd == "analyze":
        return _analyze(args)
    if args.cmd == "survey":
        return _survey(args)
    if args.cmd == "serve":
        from .server import run
        run(args.host, args.port)
        return 0
    return args.run(args)          # lab 이 단 명령 (gen / dataset / sweep)


if __name__ == "__main__":
    sys.exit(main())
```

```python
"""Stdlib-only web server: static frontend + POST /api/analyze (raw body,
filename + optional fs/fmt/dtype overrides in the query string — no multipart)."""

import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import parse_qs, urlparse

from .pipeline import analyze, survey_web
from .sigio import Meta, decode, parse_name

_WEB = Path(__file__).resolve().parent / "web"  # inside the package so wheels/sdists ship it
_MIME = {".html": "text/html", ".css": "text/css", ".js": "text/javascript",
         ".json": "application/json", ".png": "image/png", ".svg": "image/svg+xml"}
_MAX_BODY = 256 * 1024 * 1024


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *_: object) -> None:
        pass

    def _json(self, code: int, obj: dict) -> None:
        body = json.dumps(obj).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _meta(self, q: dict) -> Meta:
        """Build Meta from query params with filename fallback; raise on unknown."""
        name = q.get("name", [""])[0]
        m = parse_name(name)
        meta = Meta(float(q["fs"][0]) if "fs" in q else m.fs,
                    q.get("fmt", [m.fmt])[0], q.get("dtype", [m.dtype])[0],
                    q.get("endian", [m.endian])[0],
                    q.get("bitrev", ["1" if m.bitrev else "0"])[0] == "1",
                    rf_center=float(q["rf"][0]) if "rf" in q else m.rf_center)
        if not meta.ok():
            raise ValueError(f"샘플레이트/포맷을 알 수 없습니다: {name}")
        return meta

    def do_POST(self) -> None:  # noqa: N802 (BaseHTTPRequestHandler API)
        url = urlparse(self.path)
        if url.path not in ("/api/analyze", "/api/survey"):
            self._json(404, {"error": "not found"})
            return
        q = parse_qs(url.query)
        n = int(self.headers.get("Content-Length") or 0)
        if n <= 0:
            self._json(411, {"error": "Content-Length 필요"})
            return
        if n > _MAX_BODY:
            self._json(413, {"error": "파일이 너무 큽니다 (최대 256MB)"})
            return
        try:
            meta = self._meta(q)
            burst = int(q["burst"][0]) if "burst" in q else None
            x = decode(self.rfile.read(n), meta)
            if x.size < 256:
```

```
                raise ValueError("신호가 너무 짧습니다")
        except ValueError as exc:
            self._json(400, {"error": str(exc)})
            return
        try:
            out = survey_web(x, meta) if url.path == "/api/survey" \
                else analyze(x, meta, burst=burst).to_json()
            self._json(200, out)
        except Exception as exc:  # surface DSP failures to the UI
            self._json(500, {"error": f"분석 실패: {exc}"})

    def do_GET(self) -> None:  # noqa: N802
        rel = urlparse(self.path).path.lstrip("/") or "index.html"
        target = (_WEB / rel).resolve()
        if not (target.is_file() and target.is_relative_to(_WEB.resolve())):
            self._json(404, {"error": "not found"})
            return
        body = target.read_bytes()
        self.send_response(200)
        self.send_header("Content-Type", _MIME.get(target.suffix, "application/octet-stream"))
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)


def run(host: str = "127.0.0.1", port: int = 8000) -> None:
    srv = ThreadingHTTPServer((host, port), Handler)
    print(f"signus UI → http://{host}:{srv.server_address[1]}")
    srv.serve_forever()
```

```html
<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Signus — 신호 복조 분석기</title>
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
<link rel="stylesheet" href="/style.css">
<script defer src="/app.js"></script>
</head>
<body>
<header class="topbar">
  <div class="brand">
    <span class="logo" aria-hidden="true"></span>
    <div>
      <h1>Signus</h1>
      <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
    </div>
  </div>
</header>

<main class="wrap">
  <section id="dropCard" class="card drop" tabindex="0" role="button"
           aria-label="신호 파일 선택 또는 드래그">
    <input id="fileInput" type="file" multiple hidden>
    <div class="drop-icon" aria-hidden="true"></div>
    <p class="drop-title">신호 파일을 여기에 놓으세요</p>
    <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta</code> 지원</p>
  </section>

  <section id="metaForm" class="card form hidden">
    <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
    <div class="form-row">
      <label>샘플레이트 (Hz)
        <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
      <label>포맷
        <select id="mFmt">
          <option value="">선택</option><option value="iq">IQ</option><option value="real">Real</option>
        </select></label>
      <label>데이터 타입
        <select id="mDtype">
          <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
          <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
          <option value="f32">f32</option><option value="f64">f64</option>
        </select></label>
    </div>
    <button id="metaGo" class="btn">분석 시작</button>
  </section>

  <section id="loading" class="card loading hidden" aria-live="polite">
    <div class="spinner" aria-hidden="true"></div>
    <p id="loadMsg">신호를 분석하고 있어요…</p>
  </section>

  <section id="errorCard" class="card error hidden" role="alert">
    <div class="err-mark" aria-hidden="true">!</div>
    <div>
```

```html
        <p class="err-title">분석에 실패했어요</p>
        <p id="errMsg" class="err-msg"></p>
      </div>
    </section>

    <!-- Batch results table -->
    <section id="batchCard" class="card hidden">
      <div class="card-head">
        <h2 class="card-title">일괄 분석 결과</h2>
        <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
      </div>
      <div class="table-wrap">
        <table class="batch">
          <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th></tr></thead>
          <tbody id="batchBody"></tbody>
        </table>
      </div>
      <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
    </section>

    <!-- Wideband survey: overview waterfall with detected-emitter boxes -->
    <section id="surveyCard" class="card hidden">
      <div class="card-head">
        <h2 class="card-title">광대역 Survey</h2>
        <span id="survCount" class="pill ok"></span>
      </div>
      <div class="wf-wrap">
        <canvas id="survFall" class="fall"></canvas>
        <div id="survBoxes" class="boxes"></div>
      </div>
      <p class="hint">상자 또는 아래 목록을 클릭하면 그 신호의 상세로 이동합니다. (세로=시간, 가로=주파수)</p>
      <div id="survInfo" class="survinfo hidden"></div>
      <div class="table-wrap">
        <table class="batch">
          <thead><tr><th>중심주파수</th><th>종류</th><th>변조</th><th>심볼레이트</th><th>Lock</th></tr></thead>
          <tbody id="survBody"></tbody>
        </table>
      </div>
    </section>

    <section id="results" class="results hidden">
      <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>

      <div class="card summary">
        <div class="summary-head">
          <span id="statusPill" class="pill"></span>
          <p id="fileName" class="file-name"></p>
        </div>
        <div class="summary-body">
          <div class="gauge">
            <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
              <circle class="donut-track" cx="60" cy="60" r="52"></circle>
              <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>
            </svg>
            <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
              <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상이면 복조 신뢰">LOCK</span></div>
```

```html
        </div>
        <div class="metrics">
          <div class="metric"><span class="metric-label tip" title="변조 오차비 — 높을수록 깨끗 (dB)">MER</span>
            <span id="merVal" class="metric-val">—</span></div>
          <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 — 낮을수록 좋음 (%)">EVM</span>
            <span id="evmVal" class="metric-val">—</span></div>
          <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB)">SNR</span>
            <span id="snrVal" class="metric-val">—</span></div>
        </div>
      </div>
      <div id="flagRow" class="chips"></div>
      <div id="burstRow" class="chips"></div>
    </div>

    <div id="burstMap" class="card hidden">
      <h2 class="card-title">버스트 지도</h2>
      <div class="wf-wrap">
        <canvas id="burstFall" class="fall"></canvas>
        <div id="burstBoxes" class="boxes"></div>
      </div>
      <p class="hint">전체 녹음의 시간–주파수. 버스트 구간을 클릭하면 그 구간을 분석합니다.</p>
    </div>

    <div class="card">
      <h2 class="card-title">스펙트럼 · 워터폴</h2>
      <canvas id="specCanvas" class="spec"></canvas>
      <canvas id="fallCanvas" class="fall"></canvas>
    </div>

    <div class="card const-card">
      <div class="card-head">
        <h2 class="card-title">성상도</h2>
        <div class="play-row">
          <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏸ 일시정지</button>
          <select id="speedSel" class="sel small" aria-label="재생 속도">
            <option value="1">1x</option><option value="4" selected>4x</option>
            <option value="16">16x</option>
          </select>
        </div>
      </div>
      <canvas id="constCanvas" class="const"></canvas>
      <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
             aria-label="재생 위치">
      <p id="playInfo" class="hint"></p>
    </div>

    <div id="paramGrid" class="param-grid"></div>

    <div class="card downloads">
      <h2 class="card-title">다운로드</h2>
      <div class="dl-row">
        <button id="copyBits" class="btn ghost">비트 복사</button>
        <button id="dlBits" class="btn ghost">비트 .txt</button>
        <button id="dlSyms" class="btn ghost">심볼 .csv</button>
        <button id="dlReport" class="btn ghost">리포트 .json</button>
      </div>
```

```
    </div>
  </section>
</main>
</body>
</html>
```

```css
:root {
  --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
  --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
  --green: #12b886; --amber: #f59f00; --red: #fa5252;
  --field: #fff; --hover: #f7fafd; --border: #dfe4ea;    /* theme-sensitive surfaces */
  --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
  --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
}
/* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
    --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
    --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
    --shadow: 0 6px 24px rgba(0, 0, 0, .38);
  }
}

* { box-sizing: border-box; }

body {
  margin: 0; background: var(--bg); color: var(--ink);
  font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
  line-height: 1.5; -webkit-font-smoothing: antialiased;
}

.wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }

.topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
.brand { display: flex; align-items: center; gap: 12px; }
.logo {
  width: 40px; height: 40px; border-radius: 12px; flex: none;
  background: linear-gradient(135deg, var(--blue), #6aa8ff);
}
h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
.tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }

.card {
  background: var(--card); border-radius: var(--radius); padding: 20px;
  box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
}
.card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
.hidden { display: none !important; }

@keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform: none; } }

/* Dropzone */
.drop {
  text-align: center; cursor: pointer; border: 2px dashed var(--border);
  transition: border-color .18s, background .18s, transform .18s;
}
.drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline: none; }
.drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
.drop-icon {
  width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
  background: var(--blue-weak); position: relative;
}
.drop-icon::after {
```

```css
  content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
  border-radius: 3px; border-top: none; border-right: none;
  transform: rotate(45deg) translate(2px, -2px);
}
.drop-title { margin: 0; font-size: 16px; font-weight: 700; }
.drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }

/* Meta form */
.form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
.form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
.form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(--sub); }
input, select {
  font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
  background: var(--field); color: var(--ink);
}
input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }

.btn {
  margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;
  background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
  transition: filter .15s, transform .1s;
}
.btn:hover { filter: brightness(1.05); }
.btn:active { transform: scale(.98); }
.btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }

/* Loading */
.loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
.spinner {
  width: 26px; height: 26px; border: 3px solid var(--line);
  border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }

/* Error */
.error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
.err-mark {
  width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
  color: var(--red); font-weight: 800; display: grid; place-items: center;
}
.err-title { margin: 0; font-weight: 700; }
.err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }

/* Summary + gauge */
.summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
.pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
.pill.ok { background: var(--ok-bg); color: var(--green); }
.pill.warn { background: var(--warn-bg); color: var(--amber); }
.pill.bad { background: var(--bad-bg); color: var(--red); }
.file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }

.summary-body { display: flex; align-items: center; gap: 24px; }
.gauge { position: relative; width: 120px; height: 120px; flex: none; }
.donut { width: 120px; height: 120px; }
.donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
.donut-arc {
  fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;
```

```css
  transform: rotate(-90deg); transform-origin: center;
  transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
}
.gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
.gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nums; }
.gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }

.metrics { display: flex; flex-direction: column; gap: 12px; }
.metric { display: flex; flex-direction: column; gap: 2px; }
.metric-label { font-size: 12px; color: var(--sub); }
.tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
  text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
.metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }

/* Card head with actions: spacing lives on the CONTAINER so the title and the
   action stay vertically centered on the same line */
.card-head { display: flex; align-items: center; justify-content: space-between;
  gap: 12px; margin-bottom: 12px; }
.card-head .card-title { margin-bottom: 0; }
.btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px; }
.sel.small {
  font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
  border: 1px solid var(--border); background: var(--field); color: var(--ink);
}
.play-row { display: flex; gap: 8px; align-items: center; }
.hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }

/* Spectrum + waterfall */
.spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0d1320; }
.fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #0d1320; }

/* Constellation + playback */
.const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320; }
.scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }

/* Batch table */
.table-wrap { overflow-x: auto; }
table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
table.batch th {
  text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
  padding: 8px 10px; border-bottom: 1px solid var(--line);
}
table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
  font-variant-numeric: tabular-nums; }
table.batch .pill { padding: 3px 10px; font-size: 12px; }
table.batch tbody tr { cursor: pointer; transition: background .12s; }
table.batch tbody tr:hover { background: var(--hover); }
table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
  text-overflow: ellipsis; white-space: nowrap; }

/* Param grid */
.param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
.param { padding: 16px; margin: 0; animation: rise .4s ease both; }
.param-label { font-size: 12px; color: var(--sub); }
.param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
  font-variant-numeric: tabular-nums; }
.param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }
```

```
.chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
.chips:empty { display: none; }   /* no flags -> no ghost margin */
.chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
.chip.hit { background: var(--ok-bg); color: var(--green); }
.chip.miss { background: var(--bad-bg); color: var(--red); }
.chip.warn { background: var(--warn-bg); color: var(--amber); }

/* Burst selector (chips that act) */
.chip-btn {
  font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
  border-radius: 999px; border: 1px solid var(--border); background: var(--field);
  color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
}
.chip-btn:hover { border-color: var(--blue); color: var(--blue); }
.chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }

/* Downloads */
.dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
.dl-row .btn { flex: 1; min-width: 120px; }

/* Survey overview: waterfall + detected-emitter boxes */
.wf-wrap { position: relative; }
.wf-wrap .fall { height: 230px; border-radius: 12px; }
.boxes { position: absolute; inset: 0; pointer-events: none; }
.box {
  position: absolute; box-sizing: border-box; border: 2px solid var(--blue);
  border-radius: 5px; background: rgba(49, 130, 246, .10); cursor: pointer;
  pointer-events: auto; min-width: 7px; min-height: 12px;
  transition: background .12s, box-shadow .12s;
}
.box:hover, .box:focus-visible {
  background: rgba(49, 130, 246, .24); box-shadow: 0 0 0 2px rgba(255, 255, 255, .55); outline: none;
}
.box-lbl {   /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
  position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
  font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
  border-radius: 6px; pointer-events: none; display: none; z-index: 2;
}
.box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
.box.sel { box-shadow: 0 0 0 2px #fff, 0 0 0 4px var(--blue); }
.box.ok { border-color: var(--green); background: rgba(18, 184, 134, .13); }
.box.ok .box-lbl { background: var(--green); }
.box.warn { border-color: var(--amber); background: rgba(245, 159, 0, .13); }
.box.warn .box-lbl { background: var(--amber); }
.box.bad { border-color: var(--red); background: rgba(250, 82, 82, .13); }
.box.bad .box-lbl { background: var(--red); }
.box.gray { border-color: #8592a6; background: rgba(133, 146, 166, .15); }
.box.gray .box-lbl { background: #8592a6; }
.pill.gray { background: var(--line); color: var(--sub); }
table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }

/* Non-digital emitter inline info */
.survinfo {
  background: var(--hover); border-radius: 14px; padding: 14px 16px; margin: 12px 0;
  border-left: 4px solid #8592a6; animation: rise .3s ease both;
}
.survinfo h3 { margin: 0 0 10px; font-size: 14px; }
.si-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px 16px; }
.si-k { font-size: 11px; color: var(--sub); }
```

```
.si-v { font-size: 17px; font-weight: 700; font-variant-numeric: tabular-nums; }

@media (max-width: 420px) {
  .form-row { grid-template-columns: 1fr; }
  .param-grid { grid-template-columns: 1fr; }
  .summary-body { gap: 18px; }
}

@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

```
"use strict";
const $ = (id) => document.getElementById(id);
const API = "/api/analyze";
const SURVEY_API = "/api/survey";
const FS_RE = /(?:^|[._])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
const RF_RE = /(?:^|[._])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
                burst: null, survey: null, drilled: false, overview: null };

/* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
// baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64" };
const FMTS = { cplx: "iq", real: "real", iq: "iq" };   // iq = 옛 밑줄 형식의 이름
/* <이름>.cplx|real.<샘플레이트>.<16t>.pcm — 샘플레이트는 접두사가 없으므로 포맷 바로
   다음 자리로 찾는다. 옛 밑줄 형식(<이름>_fs…_iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
   같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
function parseName(name) {
  const base = name.replace(/^.*\//, "").toLowerCase();
  const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[._]/);
  // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
  // u8/be 오염과 점 낀 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙).
  const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
                      || t === "pcm" || t.startsWith("rf");
  const hit = (k) => /^\d+$/.test(toks[k + 1] || "") && spec(toks[k + 2] || "");
  const cands = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
  const hits = cands.filter(hit);
  const i = hits.length ? hits[hits.length - 1] : (cands.length ? cands[0] : -1);
  const tail = i >= 0 ? toks.slice(i + 1) : toks;   // 제원은 포맷 토막 뒤에만 산다
  const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
  return {
    fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
    fmt: i >= 0 ? FMTS[toks[i]] : null,
    dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
    endian: tail.includes("be") ? "be" : "le",
    bitrev: tail.includes("bitrev"),
    rf: rt ? parseFloat(rt[1]) : null,
  };
}
/* SigMF: core:datatype -> our fmt/dtype/endian */
const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
                cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
function metaFromSigmf(doc) {
  const g = doc && doc.global;
  if (!g || !g["core:datatype"]) return null;
  const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
  if (!(base in SIGMF)) return null;
  const [fmt, dtype] = SIGMF[base];
  const cap = (doc.captures || [])[0] || {};
  return { fs: Number(g["core:sample_rate"]), fmt, dtype,
           endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
           rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
}

/* --- number formatting --- */
function sig(x) {
  const a = Math.abs(x);
  return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
```

```javascript
}
function fmtHz(v) {
  if (v == null) return "—";
  const a = Math.abs(v);
  if (a >= 1e6) return sig(v / 1e6) + " MHz";
  if (a >= 1e3) return sig(v / 1e3) + " kHz";
  return sig(v) + " Hz";
}
function fmtRf(v) {   // absolute RF: keep kHz resolution even in the 100s-of-MHz range
  return (v / 1e6).toFixed(3) + " MHz";
}

/* --- ideal constellation points (mirror of constellations.ideal_points) --- */
function idealPoints(mod) {
  if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
  if (mod === "qpsk" || mod === "8psk") {
    const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
    for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(off + 2 * Math.PI * k / m) });
    return p;
  }
  const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
  for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
  for (const qi of lv) for (const ii of lv) {
    if (mod === "32qam" && Math.abs(ii * qi) === 25) continue;  // cross: corners removed
    p.push({ i: ii, q: qi });
  }
  let e = 0;
  for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
  const norm = Math.sqrt(e / p.length);
  return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
}

/* --- file intake: one data file (+sidecar) or many (batch) --- */
async function acceptFiles(list) {
  const files = [...list];
  const metas = files.filter((f) => /\.(json|sigmf-meta)$/i.test(f.name));
  const data = files.filter((f) => !metas.includes(f));
  if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
  if (data.length > 1) return runBatch(data, metas);

  state.file = data[0];
  state.resp = null;
  state.batch = [];
  state.burst = null;
  state.survey = null;
  state.drilled = false;
  state.overview = null;
  // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
  // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
  const mine = (m, ext) => {
    const stem = m.name.toLowerCase().slice(0, -ext.length);
    const dn = state.file.name.toLowerCase();
    return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
  };
  const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-meta"));
  const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
  state.sidecar = truth ? await readJson(truth) : null;   // client-side only, never sent
  state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
```

```
  if (state.meta && state.meta.fs && state.meta.fmt) runSurvey();
  else showMetaForm();
}
const readJson = (f) => f.text().then(JSON.parse).catch(() => null);

function showMetaForm() {
  hideAll();
  state.meta = state.meta || {};
  if (state.meta.fs) $("mFs").value = state.meta.fs;
  $("mFmt").value = state.meta.fmt || "";
  $("mDtype").value = state.meta.dtype || "i16";
  show($("metaForm"));
}
$("metaGo").onclick = () => {
  const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
  if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
  state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
  runSurvey();
};

/* --- server call (contract-frozen) --- */
function query(file, m, burst) {
  const q = new URLSearchParams({ name: file.name });
  if (m.fs) q.set("fs", m.fs);
  if (m.fmt) q.set("fmt", m.fmt);
  if (m.dtype) q.set("dtype", m.dtype);
  if (m.endian) q.set("endian", m.endian);
  if (m.bitrev) q.set("bitrev", "1");
  if (m.rf != null) q.set("rf", m.rf);
  if (burst != null) q.set("burst", burst);
  return q;
}
async function postTo(api, file, m, burst) {
  const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
  const data = await res.json();
  if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
  return data;
}
async function runSurvey() {                  // entry: one capture -> single detail OR survey
  hideAll();
  state.drilled = false;
  $("loadMsg").textContent = "신호를 조사하고 있어요…";
  show($("loading"));
  try {
    const resp = await postTo(SURVEY_API, state.file, state.meta);
    if (resp.mode === "single") {
      state.survey = null; state.overview = resp.overview || null; state.resp = resp.result;
      render(resp.result);
      if (state.overview) renderBurstMap(state.overview, resp.result);
    } else renderSurvey(resp);
  } catch (e) {
    showError(e.message);
  }
}
async function analyze() {                    // burst re-selection within a single signal
  hideAll();
  $("loadMsg").textContent = "신호를 분석하고 있어요…";
  show($("loading"));
  try {
    state.resp = await postTo(API, state.file, state.meta, state.burst);
```

```
    render(state.resp);
    if (state.overview) renderBurstMap(state.overview, state.resp);
    $("backBatch").classList.toggle("hidden", !state.batch.length);
  } catch (e) {
    showError(e.message);
  }
}

/* --- batch mode --- */
async function runBatch(data, metas) {
  hideAll();
  show($("loading"));
  const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
    .map((m) => [m.name.replace(/\.json$/i, ""), m]));
  const sigmfs = new Map(metas.filter((m) => /\.sigmf-meta$/i.test(m.name))
    .map((m) => [m.name.replace(/\.sigmf-meta$/i, ""), m]));
  const rows = [];
  for (let i = 0; i < data.length; i++) {
    const f = data[i];
    $("loadMsg").textContent = `일괄 분석 중… (${i + 1}/${data.length}) ${f.name}`;
    // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
    // beats filename tokens
    const sm = sigmfs.get(f.name.replace(/\.[^.]+$/, ""));
    const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
    let resp = null, err = null;
    try {
      if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
      resp = await postTo(API, f, meta);
    } catch (e) { err = e.message; }
    const tf = truths.get(f.name);
    // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
    // it used to read state.meta, which a fresh batch never set (crash) and a previous
    // single-file run left stale (silent wrong-fs decode)
    rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
  }
  state.batch = rows;
  hideAll();
  renderBatch(rows);
}

function renderBatch(rows) {
  $("batchBody").innerHTML = rows.map((r, i) => {
    if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
      `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
    const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
    const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
    return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
      `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
      `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
  }).join("");
  $("batchBody").querySelectorAll("tr").forEach((tr) => {
    tr.onclick = () => {
      const r = state.batch[+tr.dataset.i];
      if (!r.resp) return;
      state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
      state.burst = null; state.drilled = false; state.survey = null; state.overview = null;
      hideAll();
      render(r.resp);
      const b = $("backBatch");
      b.textContent = "← 목록으로";
```

```
      b.onclick = () => { hideAll(); renderBatch(state.batch); };
      b.classList.remove("hidden");
    };
  });
  show($("batchCard"));
}
$("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
$("dlCsv").onclick = () => {
  const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
  const body = state.batch.filter((r) => r.resp).map((r) => {
    const d = r.resp.detected, q = r.resp.quality;
    return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
            r.resp.snr_est_db ?? ""].join(",");
  }).join("\n");
  saveBlob("signus_batch.csv", head + body, "text/csv");
};

/* --- wideband survey overview (waterfall + detected-emitter boxes) --- */
function renderSurvey(resp) {
  hideAll();
  stopPlay();
  state.survey = resp;
  const dig = resp.emitters.filter((e) => e.result).length;
  $("survCount").textContent = `${resp.emitters.length}개 신호 · 디지털 ${dig}`;
  $("survInfo").classList.add("hidden");
  renderSurveyList(resp);
  show($("surveyCard"));                // show first so the canvas has a real width
  paintWaterfall($("survFall"), resp.overview.waterfall, 230);
  drawBoxes(resp);
}

function boxStyle(e) {   // -> [css class, short label]
  if (e.result) {
    const lock = Math.round(e.result.quality.lock);
    return [lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad",
            `${e.result.detected.mod.toUpperCase()} · ${lock}`];
  }
  if (e.kind === "chirp") {
    return ["gray", e.info && e.info.sf ? `LoRa SF${e.info.sf}` : "처프"];
  }
  return ["gray", { analog: "아날로그", tone: "톤/CW", tooshort: "짧음",
                    error: "오류" }[e.kind] || e.kind];
}

function drawBoxes(resp) {
  const host = $("survBoxes"), fs = resp.overview.fs, n = resp.overview.n || 1;
  host.innerHTML = "";
  resp.emitters.forEach((e) => {
    const d = e.det, [cls, lbl] = boxStyle(e);
    const wpct = Math.max(0.8, d.bw / fs * 100);          // width  = bandwidth / fs
    const left = (d.fc + fs / 2) / fs * 100 - wpct / 2;   // x      = carrier on -fs/2..fs/2
    const top = Math.max(0, d.t0 / n * 100);              // y      = burst start over capture
    const b = document.createElement("button");
    b.className = "box " + cls;
    b.style.left = Math.max(0, Math.min(100 - wpct, left)) + "%";
    b.style.width = wpct + "%";
    b.style.top = top + "%";
    b.style.height = Math.min(100 - top, Math.max(7, (d.t1 - d.t0) / n * 100)) + "%";
    b.title = lbl;
    b.innerHTML = `<span class="box-lbl">${lbl}</span>`;
```

```
    b.onclick = () => drillEmitter(e);
    host.appendChild(b);
  });
}

function renderSurveyList(resp) {
  const KIND = { linear: "디지털", fsk: "FSK", chirp: "처프/LoRa", analog: "아날로그",
                 tone: "톤/CW", tooshort: "짧음", error: "오류" };
  const rf0 = resp.rf_center;                      // real RF = capture centre + abs_fc
  $("survBody").innerHTML = resp.emitters.map((e, i) => {
    const [cls] = boxStyle(e), r = e.result;
    const lock = r ? `<span class="pill ${cls}">${Math.round(r.quality.lock)}</span>`
                   : `<span class="pill gray">—</span>`;
    return `<tr data-i="${i}"><td class="fc">${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs_fc)}</td>` +
      `<td>${KIND[e.kind] || e.kind}</td><td>${r ? r.detected.mod.toUpperCase() : "—"}</td>` +
      `<td>${r ? fmtHz(r.detected.baud) : "—"}</td><td>${lock}</td></tr>`;
  }).join("");
  $("survBody").querySelectorAll("tr").forEach((tr) => {
    tr.onclick = () => drillEmitter(resp.emitters[+tr.dataset.i]);
  });
}

function drillEmitter(e) {
  if (e.result) {                         // digital -> reuse the single-signal detail view
    state.drilled = true;
    state.overview = null;                // a cut channel has no whole-capture burst map
    state.resp = e.result;
    render(e.result);
    const rf0 = state.survey && state.survey.rf_center;
    $("fileName").textContent =
      `에미터 @ ${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs_fc)} · ${state.file.name}`;
    const b = $("backBatch");
    b.textContent = "← Survey로";
    b.onclick = backToSurvey;
    b.classList.remove("hidden");
  } else {                                // non-digital -> inline info, stay on the overview
    showSurveyInfo(e);
  }
}
function backToSurvey() { state.drilled = false; renderSurvey(state.survey); }

function showSurveyInfo(e) {
  const d = e.det;
  let title, rows;
  if (e.kind === "chirp" && e.info) {            // linear chirp / CSS (LoRa): characterized only
    const c = e.info;
    title = (c.sf ? `LoRa 계열 (SF${c.sf})` : "선형 처프 / CSS") + " — 복조 대상 아님";
    // rate + direction are robust (coherent from the beat tone); bw/SF only when the
    // channel is cleanly isolated enough to snap to a LoRa hypothesis.
    rows = [["중심주파수", fmtHz(e.abs_fc)],
            ["처프 방향", c.up ? "▲ 상승" : "▼ 하강"],
            ["처프율", (c.mu / 1e9).toFixed(3) + " MHz/ms"]];
    if (c.sf) rows.push(["대역폭", fmtHz(c.bw)], ["심볼레이트", fmtHz(c.rs)],
                        ["심볼시간", (c.tsym * 1e3).toFixed(2) + " ms"]);
  } else {
    const KIND = { analog: "아날로그 (FM/음성)", tone: "톤 / CW",
      tooshort: "너무 짧은 버스트", error: "분석 오류" };
    title = (KIND[e.kind] || e.kind) + " — 복조 대상 아님";
    rows = [["중심주파수", fmtHz(e.abs_fc)], ["대역폭", fmtHz(d.bw)],
```

```
      ["SNR", d.snr_db == null ? "—" : d.snr_db.toFixed(1) + " dB"],
      ["심볼레이트(추정)", fmtHz(d.baud_hint)]];
  }
  $("survInfo").innerHTML = `<h3>${title}</h3>` +
    '<div class="si-grid">' + rows.map(([k, v]) =>
      `<div><div class="si-k">${k}</div><div class="si-v">${v}</div></div>`).join("") + "</div>";
  $("survInfo").classList.remove("hidden");
  $("survInfo").scrollIntoView({ behavior: REDUCED ? "auto" : "smooth", block: "nearest" });
}

/* --- burst map: whole-record waterfall with clickable time-burst boxes (single signal) --- */
function renderBurstMap(ov, result) {
  show($("burstMap"));
  paintWaterfall($("burstFall"), ov.waterfall, 150);
  const host = $("burstBoxes"), n = ov.n || 1;
  host.innerHTML = "";
  (result.bursts || []).forEach((b, i) => {
    const top = Math.max(0, b.start / n * 100);
    const dur = (b.end - b.start) / ov.fs;
    const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0) + " ms"}`;
    const box = document.createElement("button");   // full width (one signal), spans t0..t1 in time
    box.className = "box" + (i === result.burst_idx ? " sel" : "");
    box.style.left = "0%"; box.style.width = "100%"; box.style.top = top + "%";
    box.style.height = Math.min(100 - top, Math.max(5, (b.end - b.start) / n * 100)) + "%";
    box.title = lbl;
    box.innerHTML = `<span class="box-lbl">${lbl}</span>`;
    box.onclick = () => { state.burst = i; analyze(); };   // re-analyse that burst; map persists
    host.appendChild(box);
  });
}

/* --- rendering --- */
function statusOf(lock) {
  if (lock >= 60) return ["복조 성공", "ok"];
  if (lock >= 40) return ["부분 복조", "warn"];
  return ["복조 실패", "bad"];
}
function render(d) {
  hideAll();
  show($("results"));
  $("burstMap").classList.add("hidden");   // re-shown by renderBurstMap only in multi-burst single mode
  const lock = d.quality.lock, [txt, cls] = statusOf(lock);
  const pill = $("statusPill");
  pill.textContent = txt;
  pill.className = "pill " + cls;
  $("fileName").textContent =
    `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
  $("merVal").textContent = d.quality.mer_db == null ? "—" : d.quality.mer_db.toFixed(1) + " dB";
  $("evmVal").textContent = d.quality.evm == null ? "—" : (d.quality.evm * 100).toFixed(1) + " %";
  $("snrVal").textContent = snrText(d);

  const flags = [];
  if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
  if (d.eq && d.eq.applied) flags.push(`<span class="chip hit">등화기 적용 · ${
    d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"}</span>`);
  if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>');
  if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>');
  if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어싱 가능<
```

```
    /span>');
  $("flagRow").innerHTML = flags.join("");

  renderBursts(d);
  animateGauge(lock, cls);
  renderParams(d);
  drawSpectrum(d);
  drawWaterfall(d);
  startPlay(d);
}
function snrText(d) {
  const s = d.snr_est_db;
  if (s == null || d.quality.lock < 30) return "—";
  return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
}

function renderBursts(d) {
  const row = $("burstRow"), bs = d.bursts || [];
  // a survey-drilled emitter is already a cut channel: burst re-selection would
  // re-post the whole capture, so it is disabled while drilled. When the burst MAP
  // is shown (multi-burst single signal) the map replaces the chips.
  if (state.drilled || state.overview || bs.length < 2) { row.innerHTML = ""; return; }
  const dur = (b) => {
    const sec = (b.end - b.start) / d.fs;
    return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
  };
  row.innerHTML = bs.map((b, i) =>
    `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
    `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
  row.querySelectorAll("button").forEach((el) => {
    el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
  });
}

function chip(hit, truthTxt) {
  return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</span>`;
}
function renderParams(d) {
  const det = d.detected, t = state.sidecar && state.sidecar.truth;
  const near = (a, b, tol) => Math.abs(a - b) <= tol;
  const rows = [
    ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
      t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
      det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
    ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.baud), fmtHz(t.baud)),
      det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
    ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUpperCase()),
      det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
  ];
  if (d.family === "fsk") {
    rows.push(["변조지수 h", det.h.toFixed(2),
      t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
  } else {
    rows.push(["롤오프", det.rolloff.toFixed(2),
      t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2)), ""]);
  }
```

```
  $("paramGrid").innerHTML = rows.map(([label, val, hit, note]) => `
    <div class="card param">
      <div class="param-label">${label}</div>
      <div class="param-val">${val}</div>
      ${note ? `<div class="param-note">${note}</div>` : ""}
      ${hit ? `<div class="chips">${hit}</div>` : ""}
    </div>`).join("");
}

function animateGauge(lock, cls) {
  const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
  arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
  arc.style.strokeDasharray = C;
  const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
  if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return; }
  arc.style.strokeDashoffset = C;
  requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
  const t0 = performance.now();
  const step = (t) => {
    const k = Math.min(1, (t - t0) / 900);
    num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
    if (k < 1) requestAnimationFrame(step);
  };
  requestAnimationFrame(step);
}

/* --- canvas helpers --- */
function fit(cv, hCss) {
  const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
  const h = hCss || w;
  cv.width = w * dpr; cv.height = h * dpr;
  const g = cv.getContext("2d");
  g.setTransform(dpr, 0, 0, dpr, 0, 0);
  return [g, w, h];
}
function pct(a, p) {
  const s = [...a].sort((x, y) => x - y);
  return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
}

/* --- spectrum --- */
function drawSpectrum(d) {
  const [g, w, h] = fit($("specCanvas"), 120);
  g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
  if (!d.spectrum) return;
  const f = d.spectrum.f, db = d.spectrum.db;
  const lo = pct(db, 0.02), hi = pct(db, 1) + 2;
  const fmin = f[0], fmax = f[f.length - 1];
  const X = (v) => (v - fmin) / (fmax - fmin) * w;
  const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
  const det = d.detected, half = det.baud / 2e3;
  g.strokeStyle = "rgba(120,180,255,.35)"; g.setLineDash([4, 4]);
  for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
    g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
  }
  g.setLineDash([]);
  g.strokeStyle = "rgba(255,255,255,.45)";
  g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
  g.strokeStyle = "#5aa9ff"; g.lineWidth = 1.4; g.beginPath();
  f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
```

```
  g.stroke();
  g.fillStyle = "rgba(255,255,255,.5)"; g.font = "10px sans-serif";
  g.fillText(sig(fmin) + " kHz", 4, h - 4);
  const tmax = sig(fmax) + " kHz";
  g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
}

/* --- waterfall --- */
function drawWaterfall(d) { paintWaterfall($("fallCanvas"), d.waterfall, 150); }
function paintWaterfall(canvas, wf, hCss) {
  const [g, w, h] = fit(canvas, hCss);
  g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
  if (!wf || !wf.rows) return;
  const lo = pct(wf.db, 0.05), hi = pct(wf.db, 0.95);
  const img = g.createImageData(wf.bins, wf.rows);
  for (let k = 0; k < wf.db.length; k++) {
    const t = Math.max(0, Math.min(1, (wf.db[k] - lo) / (hi - lo + 1e-9)));
    const o = k * 4;                              // dark -> blue -> white ramp
    img.data[o] = t < 0.5 ? 13 + t * 120 : 73 + (t - 0.5) * 364;
    img.data[o + 1] = t < 0.5 ? 19 + t * 260 : 149 + (t - 0.5) * 212;
    img.data[o + 2] = t < 0.5 ? 32 + t * 446 : 255;
    img.data[o + 3] = 255;
  }
  createImageBitmap(img).then((bmp) => { g.imageSmoothingEnabled = true; g.drawImage(bmp, 0, 0, w, h); });
}

/* --- constellation playback (the headline) --- */
const play = { raf: 0, i: 0, on: false, data: null };
let CG = null;   // cached constellation geometry

function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
function setPlayBtn(on) { $("playBtn").textContent = on ? "❚❚ 일시정지" : "▶ 재생"; }

function startPlay(d) {
  stopPlay();
  play.data = d;
  play.i = 0;
  const n = d.constellation.i.length;
  $("scrub").max = String(Math.max(1, n));
  $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
  drawConstBase(d);
  if (REDUCED) {                         // static full view, no motion
    drawPoints(d, 0, n);
    $("scrub").value = String(n);
    setPlayBtn(false);
    return;
  }
  play.on = true;
  setPlayBtn(true);
  play.raf = requestAnimationFrame(loop);
}
$("playBtn").onclick = () => {
  if (!play.data) return;
  if (play.on) { stopPlay(); setPlayBtn(false); }
  else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
};
$("scrub").oninput = () => {
  if (!play.data) return;
  stopPlay(); setPlayBtn(false);
```

```
  play.i = +$("scrub").value;
  drawConstBase(play.data);
  drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
};

function loop() {
  const d = play.data, n = d.constellation.i.length;
  const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
  const to = Math.min(n, play.i + step);
  fade();                              // phosphor persistence
  drawPoints(d, play.i, to);
  play.i = to >= n ? 0 : to;
  if (play.i === 0) drawConstBase(d);  // loop: clear the trail
  $("scrub").value = String(play.i);
  if (play.on) play.raf = requestAnimationFrame(loop);
}

function drawConstBase(d) {
  const [g, w] = fit($("constCanvas"));
  const fsk = d.family === "fsk";
  const I = d.constellation.i, Q = d.constellation.q;
  const ideal = fsk ? [] : idealPoints(d.detected.mod);
  let lim = 0;
  for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
  for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
  lim = (lim || 1) * 1.12;
  const R = w / 2 * 0.9;
  CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };

  g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
  g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
  for (let f = -1; f <= 1; f += 0.5) {
    const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
    g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
    g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
  }
  if (fsk) {                             // frequency levels as dashed bands
    g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
    for (const lv of [...new Set(I.map((v) => Math.round(v)))].filter((v) => Math.abs(v) <= 8)) {
      const [, yy] = CG.map(0, lv);
      g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
    }
    g.setLineDash([]);
  } else {
    g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
    for (const p of ideal) {
      const [x, y] = CG.map(p.i, p.q);
      g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
    }
  }
}
function fade() {
  if (!CG) return;
  CG.g.fillStyle = "rgba(13,19,32,.14)";   // translucent wash = phosphor decay
  CG.g.fillRect(0, 0, CG.w, CG.w);
}
function drawPoints(d, from, to) {
  if (!CG || to <= from) return;
  const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
  const fsk = d.family === "fsk";
```

```
  const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
  g.globalCompositeOperation = "lighter";
  g.fillStyle = `rgba(90,170,255,${a})`;
  for (let k = from; k < to; k++) {
    // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
    const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
    g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
  }
  g.globalCompositeOperation = "source-over";
}

/* --- downloads --- */
function saveBlob(name, text, type) {
  const url = URL.createObjectURL(new Blob([text], { type }));
  const a = document.createElement("a");
  a.href = url; a.download = name; a.click();
  URL.revokeObjectURL(url);
}
const stem = () => state.file.name.replace(/\.[^.]*$/, "");
$("copyBits").onclick = async (e) => {   // decoded bitstream -> clipboard, with brief feedback
  const btn = e.currentTarget, was = btn.textContent;
  try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
  catch { btn.textContent = "복사 실패"; }
  setTimeout(() => { btn.textContent = was; }, 1400);
};
$("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
$("dlSyms").onclick = () => {
  const c = state.resp.constellation;
  saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/csv");
};
$("dlReport").onclick = () =>
  saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");

/* --- view helpers --- */
function show(el) { el.classList.remove("hidden"); }
function hideAll() {
  stopPlay();
  ["metaForm", "loading", "errorCard", "results", "batchCard", "surveyCard"]
    .forEach((id) => $(id).classList.add("hidden"));
  $("backBatch").classList.add("hidden");
}
function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }

/* --- dropzone wiring --- */
const drop = $("dropCard"), input = $("fileInput");
// reset value first: re-picking the SAME file fires no change event otherwise
const pick = () => { input.value = ""; input.click(); };
drop.onclick = pick;
drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick(); } };
input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
drop.addEventListener("drop", (e) => {
  e.preventDefault(); drop.classList.remove("drag");
  if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
});
```